Name: Aryan Sunil Bendre
Student: 5144984
Roll NO: 03
College: B. K. Birla College

Practical 1

1. Write programs to parse text files, CSV, HTML, XML and JSON documents and extract relevant data. After retrieving data check any anomalies in the data, missing values etc.

In [ ]:
```
import pandas as pd
import numpy as np
import os

# ========================================
# 0. SETUP: CREATE DUMMY FILES FOR TESTING
# ========================================
# This generates sample files in Colab so you can test the code immediate

csv_data = """id,name,age,salary
1,Alice,28,50000
2,Bob,35,
3,Charlie,,70000
1,Alice,28,50000""" # Intentional missing values and duplicate Alice

with open('sample.csv', 'w') as f:
    f.write(csv_data)

json_data = """[
    {"id": 1, "name": "Alice", "age": 28, "salary": 50000},
    {"id": 2, "name": "Bob", "age": 35, "salary": null},
    {"id": 3, "name": "Charlie", "age": null, "salary": 70000}
]"""
with open('sample.json', 'w') as f:
    f.write(json_data)

html_data = """
<table>
  <tr><th>id</th><th>name</th><th>age</th><th>salary</th></tr>
  <tr><td>1</td><td>Alice</td><td>28</td><td>50000</td></tr>
  <tr><td>2</td><td>Bob</td><td>35</td><td></td></tr>
</table>
"""
with open('sample.html', 'w') as f:
    f.write(html_data)

xml_data = """<?xml version="1.0" ?>
```

```
<data>
    <record><id>1</id><name>Alice</name><age>28</age><salary>50000</salar
    <record><id>2</id><name>Bob</name><age>35</age><salary></salary></rec
</data>
"""
with open('sample.xml', 'w') as f:
    f.write(xml_data)

# ==========================================
# 1. PARSING FUNCTIONS
# ==========================================

def parse_csv(file_path):
    print(f"\n--- Parsing CSV: {file_path} ---")
    return pd.read_csv(file_path)

def parse_json(file_path):
    print(f"\n--- Parsing JSON: {file_path} ---")
    return pd.read_json(file_path)

def parse_html(file_path):
    print(f"\n--- Parsing HTML: {file_path} ---")
    tables = pd.read_html(file_path)
    return tables[0] # Returns the first table found in the HTML

def parse_xml(file_path):
    print(f"\n--- Parsing XML: {file_path} ---")
    return pd.read_xml(file_path)

def parse_text(file_path, delimiter='\t'):
    print(f"\n--- Parsing Text File: {file_path} ---")
    return pd.read_csv(file_path, delimiter=delimiter)


# ==========================================
# 2. DATA QUALITY & ANOMALY DETECTION
# ==========================================

def check_data_anomalies(df, dataset_name="Dataset"):
    """
    Analyzes a DataFrame for missing values, duplicates, and basic anomal
    """
    print(f"\n[{dataset_name}] Data Quality Report:")
    print("-" * 50)

    if df is None or df.empty:
        print("Dataframe is empty or invalid.")
        return

    # 1. Check for Missing Values (Nulls/NaNs)
    missing_values = df.isnull().sum()
    total_missing = missing_values.sum()
    if total_missing > 0:
        print(f"⚠ Missing Values Found:\n{missing_values[missing_values >
```

```
    else:
        print("✅ No missing values found.\n")

    # 2. Check for Duplicate Rows
    duplicates = df.duplicated().sum()
    if duplicates > 0:
        print(f"⚠️ Found {duplicates} duplicate row(s).\n")
    else:
        print("✅ No duplicate rows found.\n")

    # 3. Show Data Types
    print("Data Types:")
    print(df.dtypes)
    print("\n" + "=" * 50)


# ==========================================
# 3. MAIN EXECUTION
# ==========================================
if __name__ == "__main__":

    # 1. Test CSV
    df_csv = parse_csv('sample.csv')
    check_data_anomalies(df_csv, "CSV Data")

    # 2. Test JSON
    df_json = parse_json('sample.json')
    check_data_anomalies(df_json, "JSON Data")

    # 3. Test HTML
    df_html = parse_html('sample.html')
    check_data_anomalies(df_html, "HTML Data")

    # 4. Test XML
    df_xml = parse_xml('sample.xml')
    check_data_anomalies(df_xml, "XML Data")
```

```
--- Parsing CSV: sample.csv ---

[CSV Data] Data Quality Report:
--------------------------------------------------
⚠ Missing Values Found:
age       1
salary    1
dtype: int64

⚠ Found 1 duplicate row(s).

Data Types:
id          int64
name       object
age       float64
salary    float64
dtype: object

==================================================

--- Parsing JSON: sample.json ---

[JSON Data] Data Quality Report:
--------------------------------------------------
⚠ Missing Values Found:
age       1
salary    1
dtype: int64

✅ No duplicate rows found.

Data Types:
id          int64
name       object
age       float64
salary    float64
dtype: object

==================================================

--- Parsing HTML: sample.html ---

[HTML Data] Data Quality Report:
--------------------------------------------------
⚠ Missing Values Found:
salary    1
dtype: int64

✅ No duplicate rows found.

Data Types:
id          int64
name       object
age         int64
salary    float64
```

```
dtype: object

==================================================

--- Parsing XML: sample.xml ---

[XML Data] Data Quality Report:
--------------------------------------------------
⚠ Missing Values Found:
salary    1
dtype: int64

✅ No duplicate rows found.

Data Types:
id          int64
name       object
age         int64
salary    float64
dtype: object

==================================================
```

2. Write programs for reading and writing binary files

In [ ]:

```python
import pandas as pd
import pickle
import os

# ==========================================
# 1. RAW BINARY READ/WRITE FUNCTIONS
# ==========================================

def write_raw_binary(file_path, byte_data):
    """Writes raw byte data to a binary file."""
    print(f"Writing raw binary data to '{file_path}'...")
    # 'wb' stands for Write Binary
    with open(file_path, 'wb') as file:
        file.write(byte_data)
    print("✅ Write successful.\n")

def read_raw_binary(file_path):
    """Reads raw byte data from a binary file."""
    print(f"Reading raw binary data from '{file_path}'...")
    # 'rb' stands for Read Binary
    with open(file_path, 'rb') as file:
        data = file.read()
    return data

# ==========================================
# 2. PICKLE BINARY (OBJECT SERIALIZATION)
# ==========================================

def write_pickle_binary(file_path, data_object):
```

```
    """Serializes a Python object (like a DataFrame) to a binary file."""
    print(f"Writing DataFrame to binary pickle file '{file_path}'...")
    with open(file_path, 'wb') as file:
        pickle.dump(data_object, file)
    print("✅ Pickling successful.\n")

def read_pickle_binary(file_path):
    """Deserializes a Python object from a binary pickle file."""
    print(f"Reading DataFrame from binary pickle file '{file_path}'...")
    with open(file_path, 'rb') as file:
        data_object = pickle.load(file)
    return data_object

# ==========================================
# 3. MAIN EXECUTION (Colab Ready)
# ==========================================
if __name__ == "__main__":

    print("=== PART 1: Raw Binary Operations ===")
    raw_file = 'raw_data.bin'

    # Prefixing a string with 'b' converts it to raw bytes
    my_binary_data = b"Data Engineering is awesome! \x00\x01\x02\x03"

    # Write and Read
    write_raw_binary(raw_file, my_binary_data)
    retrieved_raw_data = read_raw_binary(raw_file)

    print(f"Retrieved Bytes: {retrieved_raw_data}")
    print("-" * 50)


    print("\n=== PART 2: DataFrame Pickling Operations ===")
    pickle_file = 'dataframe_data.pkl'

    # Create a sample pandas DataFrame
    df = pd.DataFrame({
        'emp_id': [101, 102, 103],
        'name': ['Alice', 'Bob', 'Charlie'],
        'department': ['Data', 'Engineering', 'Analytics']
    })

    # Write and Read
    write_pickle_binary(pickle_file, df)
    retrieved_df = read_pickle_binary(pickle_file)

    print("Retrieved DataFrame from Binary:")
    print(retrieved_df)
    print("-" * 50)
```

```
=== PART 1: Raw Binary Operations ===
Writing raw binary data to 'raw_data.bin'...
✅ Write successful.

Reading raw binary data from 'raw_data.bin'...
Retrieved Bytes: b'Data Engineering is awesome! \x00\x01\x02\x03'
--------------------------------------------------

=== PART 2: DataFrame Pickling Operations ===
Writing DataFrame to binary pickle file 'dataframe_data.pkl'...
✅ Pickling successful.

Reading DataFrame from binary pickle file 'dataframe_data.pkl'...
Retrieved DataFrame from Binary:
   emp_id     name   department
0     101    Alice         Data
1     102      Bob  Engineering
2     103  Charlie    Analytics
--------------------------------------------------
```

3. Write programs for searching, splitting, and replacing strings based on pattern matching using regular expressions

In [ ]:

```python
import re

text_data = "Contact support at support@example.com or sales at sales123@

# 1. Searching (Extracting Emails)
# Pattern looks for characters, an @ symbol, characters, a dot, and a dom
email_pattern = r'[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+'
emails_found = re.findall(email_pattern, text_data)
print("Searched Emails:", emails_found)
# Output: ['support@example.com', 'sales123@example.org']

# 2. Splitting (Split string by spaces OR punctuation)
# Pattern splits on any whitespace or standard punctuation marks
split_pattern = r'[\s,.]+'
words = re.split(split_pattern, text_data)
# Filter out empty strings from the result
words = [w for w in words if w]
print("Split Words:", words)

# 3. Replacing (Anonymize phone numbers)
# Pattern looks for a standard 3-4 digit phone format and replaces it
phone_pattern = r'\d{3}-\d{4}'
anonymized_text = re.sub(phone_pattern, 'XXX-XXXX', text_data)
print("Replaced Text:", anonymized_text)
# Output: "...Call XXX-XXXX."
```

```
Searched Emails: ['support@example.com', 'sales123@example.org.']
Split Words: ['Contact', 'support', 'at', 'support@example', 'com', 'or',
'sales', 'at', 'sales123@example', 'org', 'Call', '555-1234']
Replaced Text: Contact support at support@example.com or sales at sales12
3@example.org. Call XXX-XXXX.
```

4. Design a relational database for a small application and populate the database. Using SQL do the CRUD (create, read, update and delete) operation

In [ ]:

```
import sqlite3

# Connect to database (creates the file if it doesn't exist)
conn = sqlite3.connect('company.db')
cursor = conn.cursor()

# DESIGN: Create Table
cursor.execute('''
    CREATE TABLE IF NOT EXISTS employees (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        department TEXT NOT NULL,
        salary REAL
    )
''')
conn.commit()

# CREATE: Insert Data
employees_data = [
    ('Alice Smith', 'Engineering', 85000.00),
    ('Bob Jones', 'Marketing', 62000.00),
    ('Charlie Brown', 'Sales', 55000.00)
]
cursor.executemany('''
    INSERT INTO employees (name, department, salary)
    VALUES (?, ?, ?)
''', employees_data)
conn.commit()
print("Data Inserted.")

# READ: Retrieve Data
cursor.execute("SELECT * FROM employees WHERE salary > 60000")
high_earners = cursor.fetchall()
print("\nHigh Earners (Read):")
for emp in high_earners:
    print(emp)

# UPDATE: Modify Existing Data
cursor.execute('''
    UPDATE employees
    SET salary = 58000.00
    WHERE name = 'Charlie Brown'
''')
conn.commit()
print("\nRecord Updated.")

# DELETE: Remove Data
cursor.execute('''
```

```
    DELETE FROM employees
    WHERE department = 'Marketing'
''')
conn.commit()
print("Record Deleted.")

# Verify Final State
cursor.execute("SELECT * FROM employees")
print("\nFinal Database State:")
for row in cursor.fetchall():
    print(row)

# Close the connection
conn.close()
```

```
Data Inserted.

High Earners (Read):
(1, 'Alice Smith', 'Engineering', 85000.0)
(4, 'Alice Smith', 'Engineering', 85000.0)
(7, 'Alice Smith', 'Engineering', 85000.0)
(10, 'Alice Smith', 'Engineering', 85000.0)
(11, 'Bob Jones', 'Marketing', 62000.0)

Record Updated.
Record Deleted.

Final Database State:
(1, 'Alice Smith', 'Engineering', 85000.0)
(3, 'Charlie Brown', 'Sales', 58000.0)
(4, 'Alice Smith', 'Engineering', 85000.0)
(6, 'Charlie Brown', 'Sales', 58000.0)
(7, 'Alice Smith', 'Engineering', 85000.0)
(9, 'Charlie Brown', 'Sales', 58000.0)
(10, 'Alice Smith', 'Engineering', 85000.0)
(12, 'Charlie Brown', 'Sales', 58000.0)
```

Name: Aryan Sunil Bendre
Student ID: 5144984
Roll NO: 03
Subject: Data Engineering

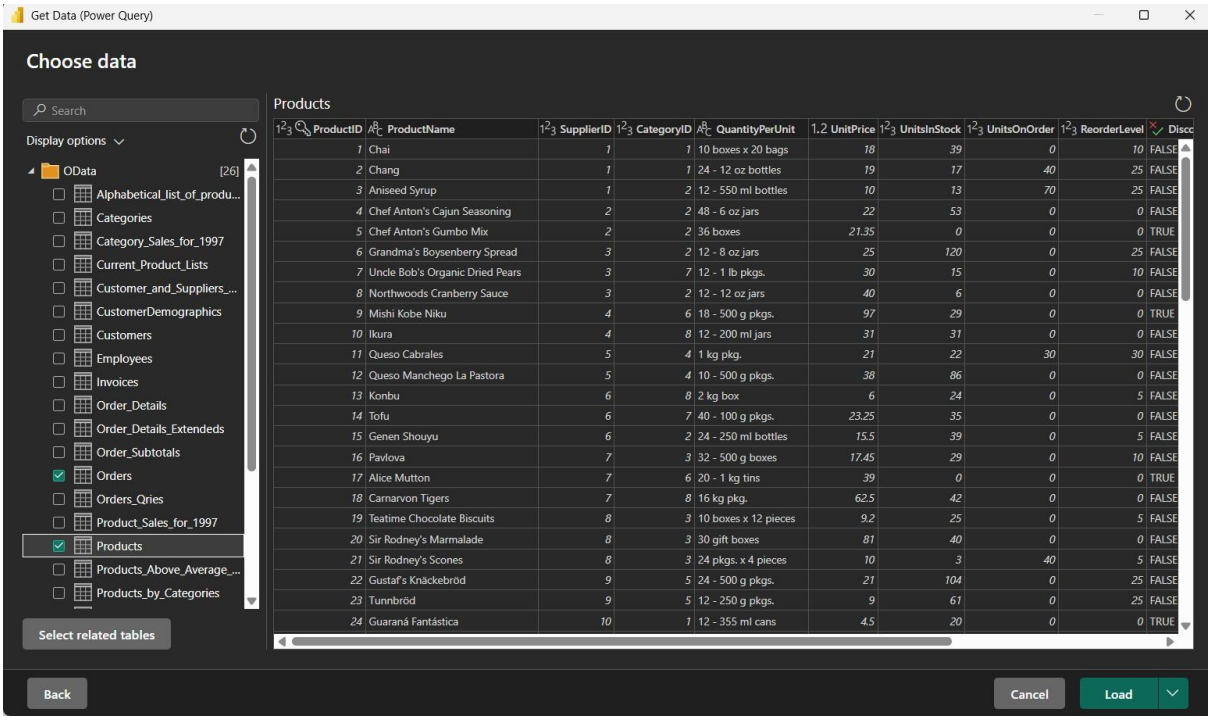
Practical No. – 2

Aim:

To perform the ETL process in Power BI by extracting, transforming, and loading data to prepare it for analysis.

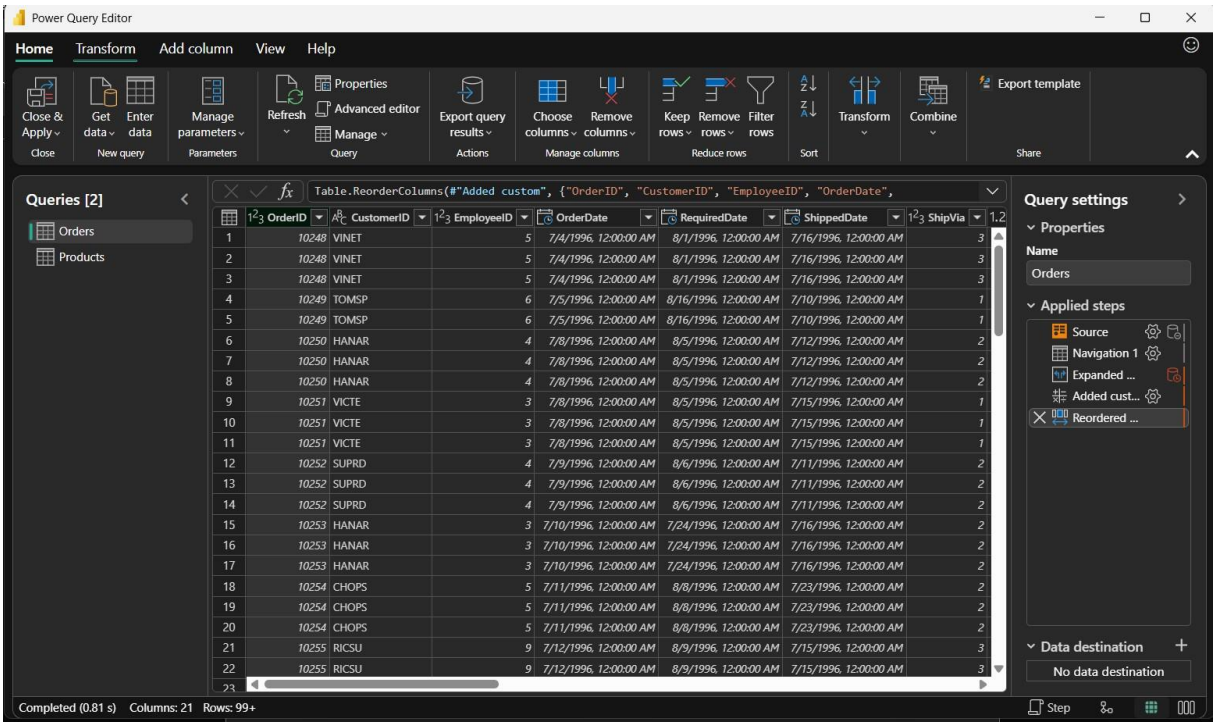
Step 1- Import Excel Data into Power BI

The Excel workbook containing the Products, Orders, and Order_Details tables was imported into Power BI for further data transformation and analysis.



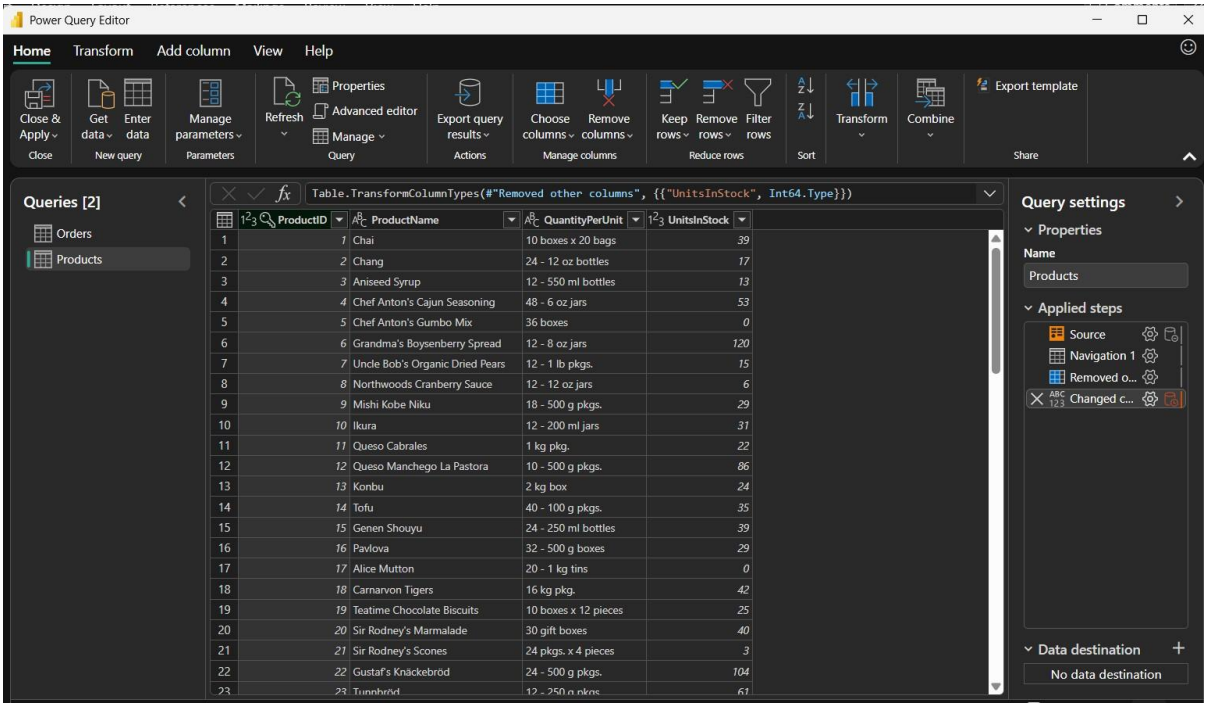
Step 2- Open Power Query Editor

Power Query Editor was opened to perform the required data transformation operations on the imported tables.



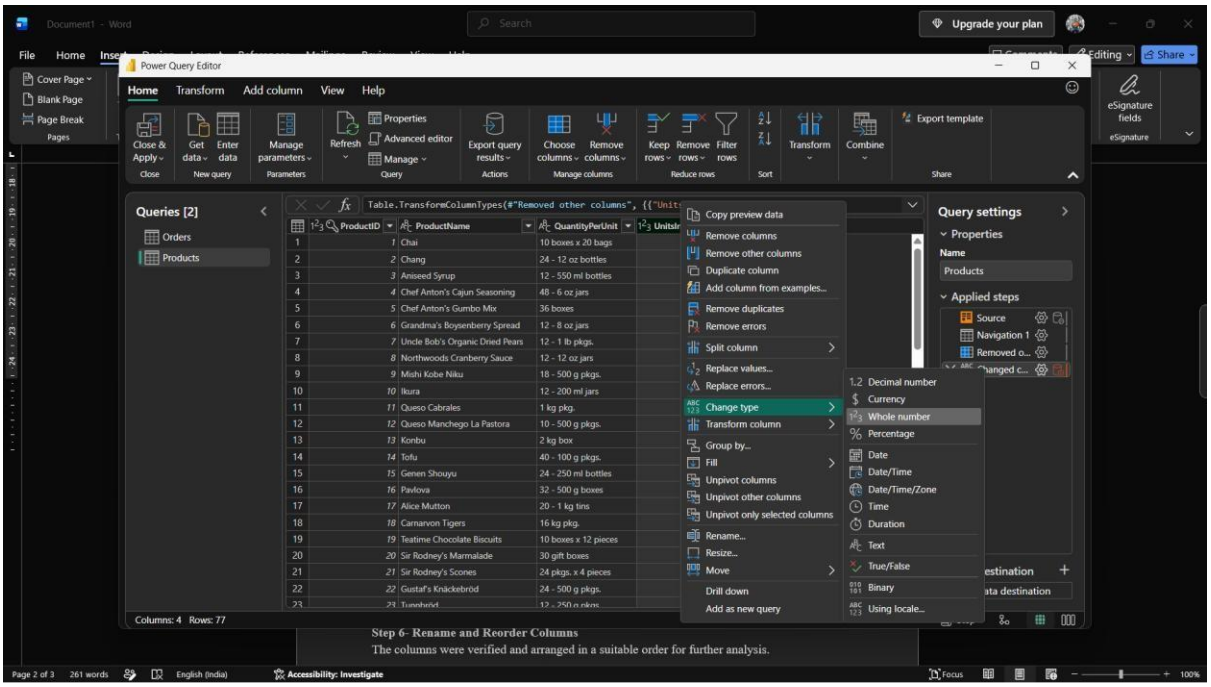
Step 3- Remove Unnecessary Columns

Unnecessary columns were removed from the Products table to retain only the fields required for further analysis.



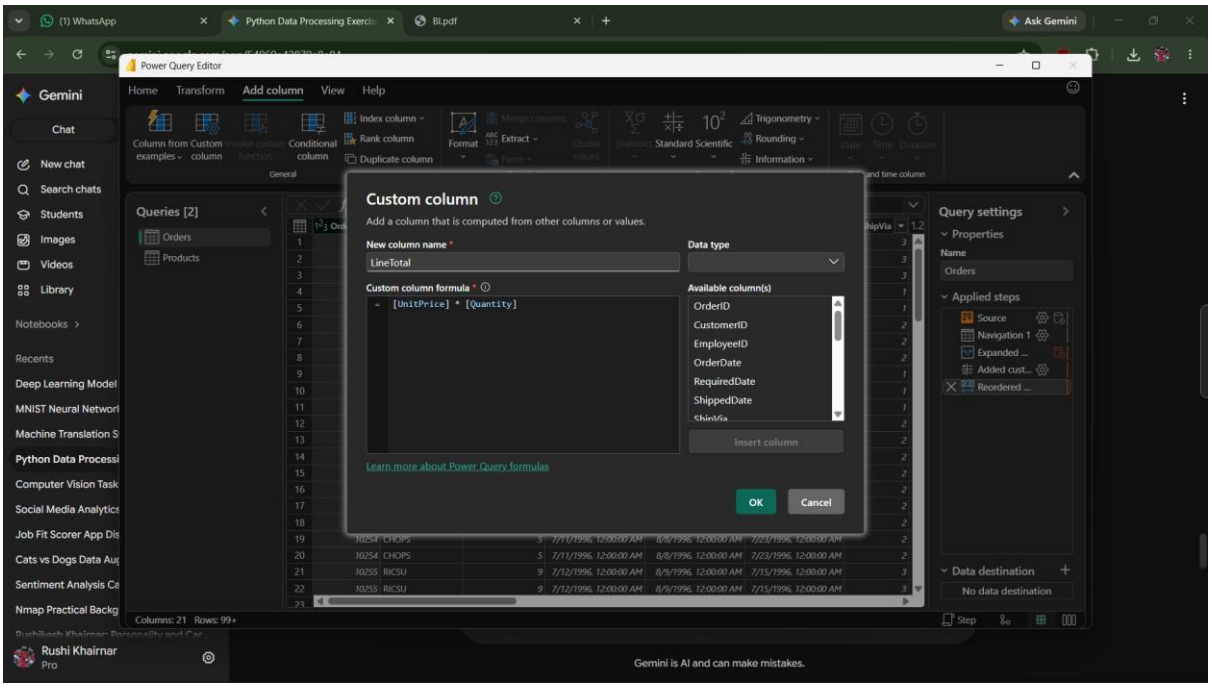
Step 4 - Verify the Data Type of UnitsInStock

The data type of the UnitsInStock column was verified as Whole Number, which is appropriate for representing stock quantities.



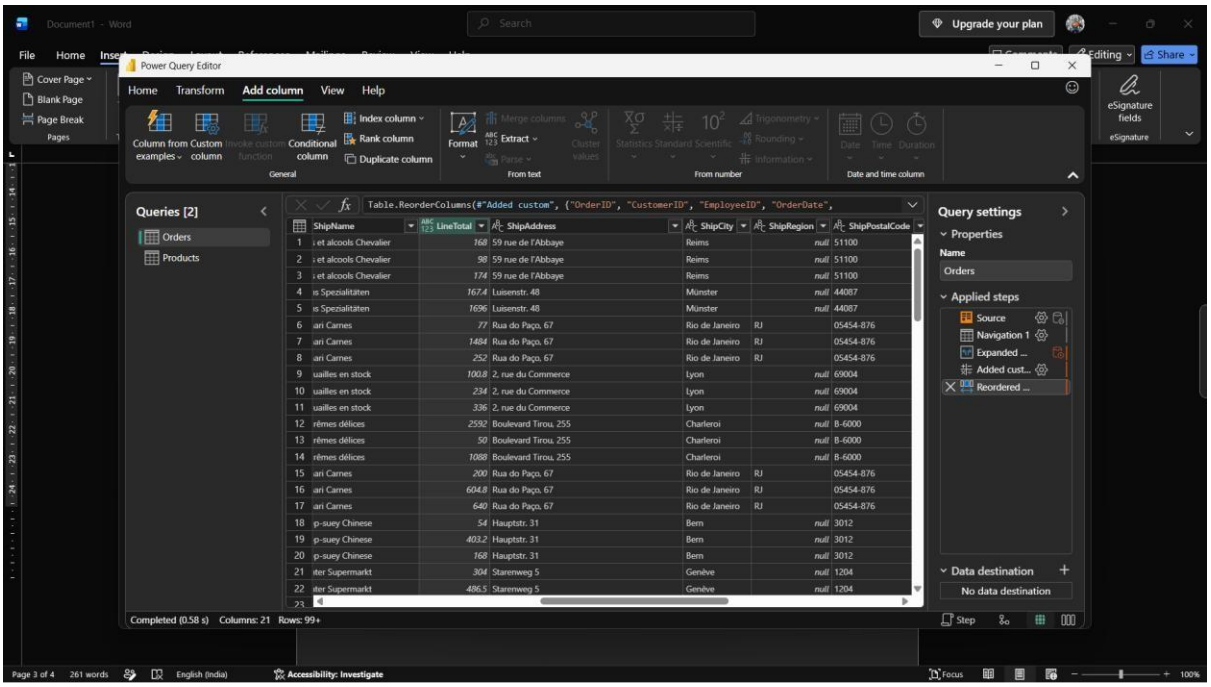
Step 5- Calculate Line Total

A new LineTotal column was created by multiplying UnitPrice by Quantity for each order detail.



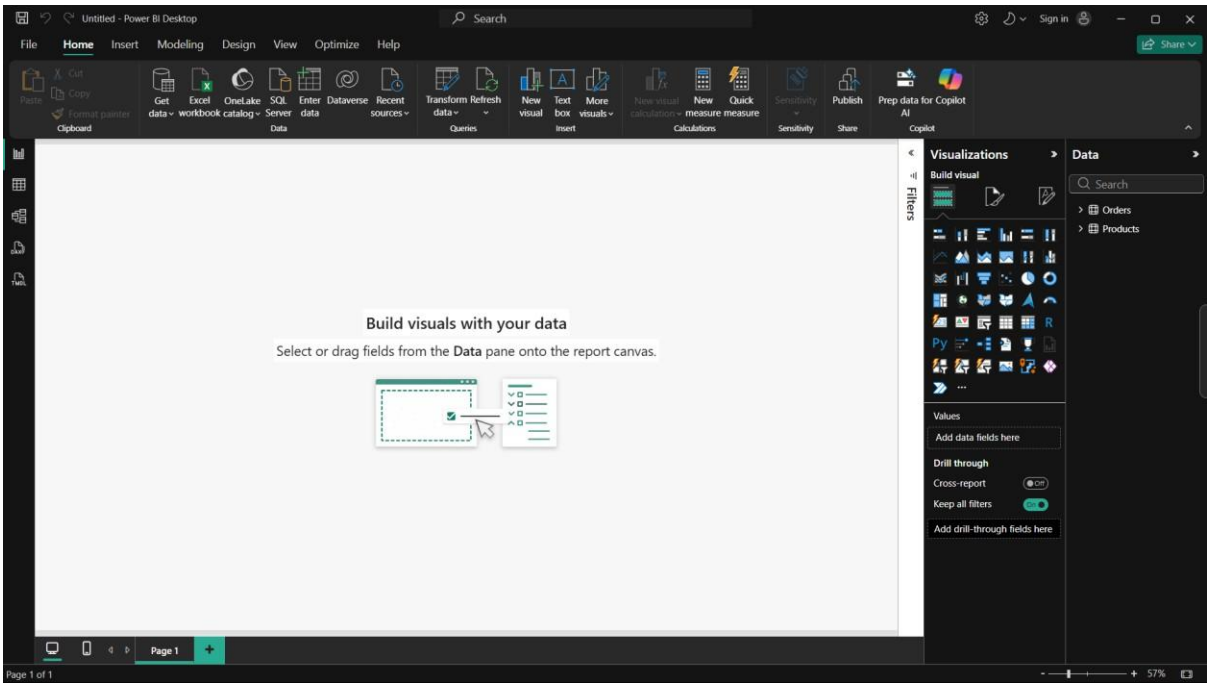
Step 6- Rename and Reorder Columns

The columns were verified and arranged in a suitable order for further analysis.



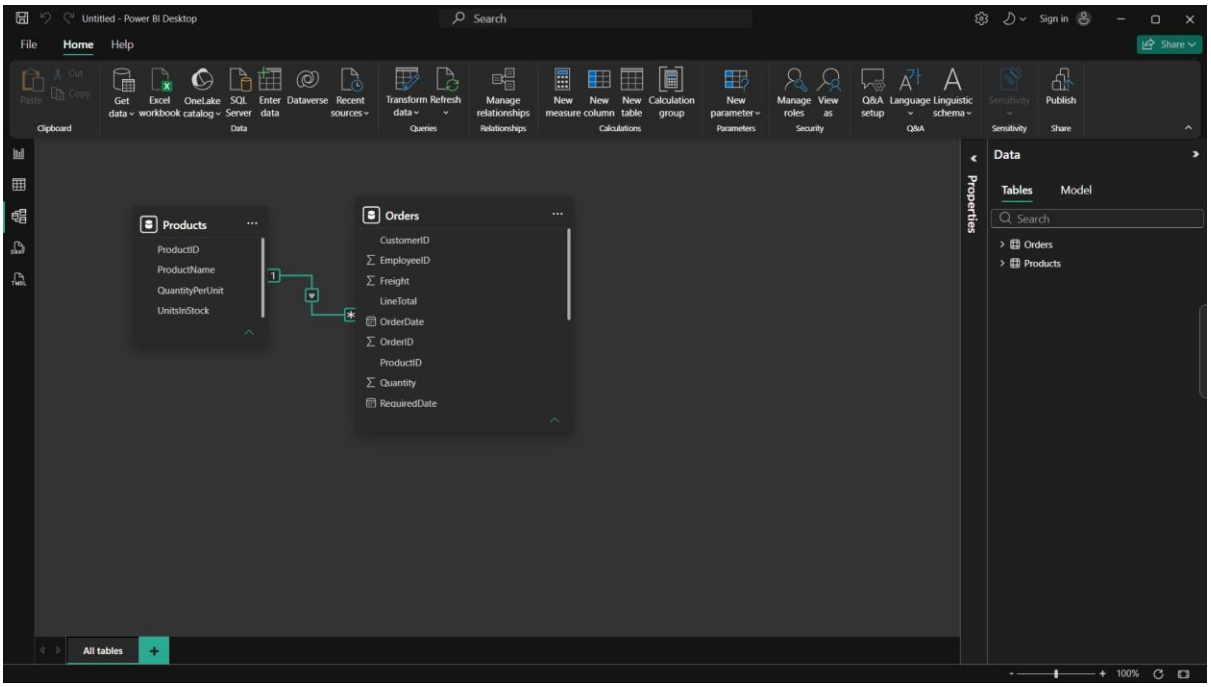
Step 7- Load the Transformed Data

The transformed data was loaded into Power BI Desktop using the Close & Apply option.



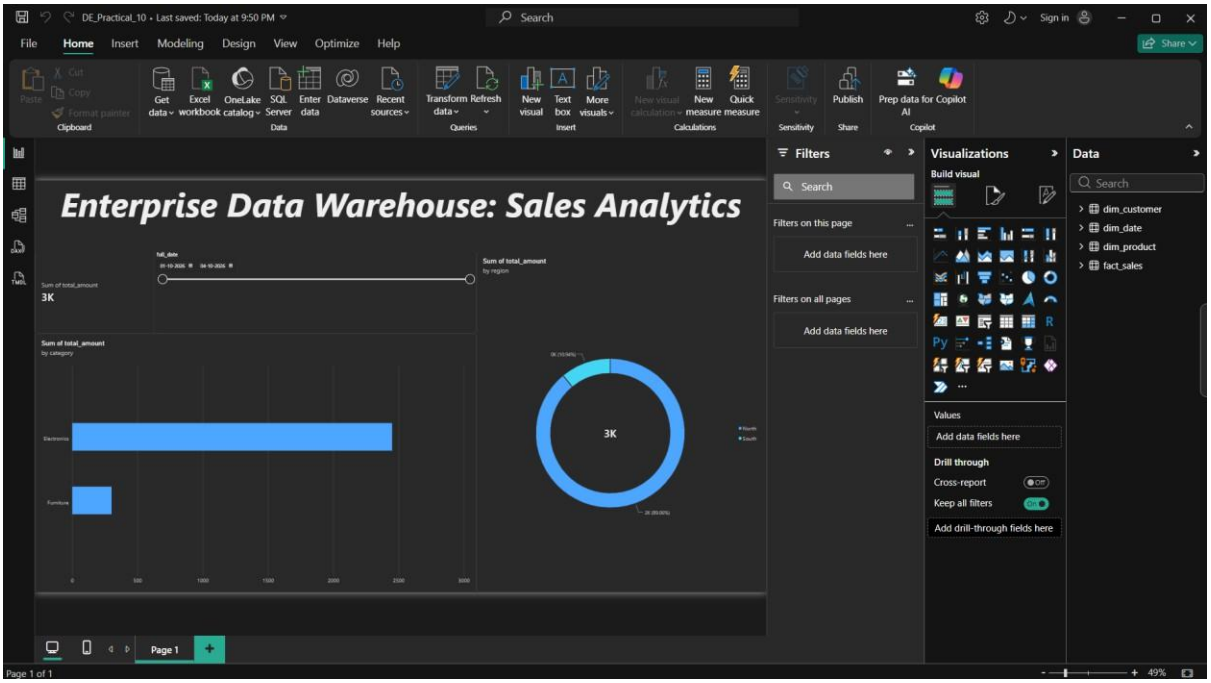
Step 8 Create Relationships Between Tables

Relationships were created between the related tables using their common key fields to establish the data model.



Step 9 Result

The ETL process was successfully performed in Power BI by extracting data from an Excel workbook, transforming the data using Power Query, creating the Line Total column, loading the transformed data, and establishing relationships between the tables.



Name: Aryan Sunil Bendre
Student ID: 5144984
Roll NO: 03
Subject: Data Engineering

Practical 3: MongoDB Operations

Aim:
To perform single and multiple document insertion and retrieval using MongoDB shell (mongosh).

Step 1: Insert Single Document
Query:
db.items.insertOne({ name: "laptop", price: 999 });

Output:

```
>_MONGOSH
> db.items.insertOne({ name: "laptop", price: 999 });
< {
    acknowledged: true,
    insertedId: ObjectId('6ac649038fa08d8f4561c384')
  }
test>
```

Step 2: View Inserted Document
Query:
db.items.find();

Output:

```
> db.items.find();
< {
    _id: ObjectId('6ac51e1cd4bbf249c034346a'),
    name: 'laptop',
    price: 999
  }
  {
    _id: ObjectId('6ac649038fa08d8f4561c384'),
    name: 'laptop',
    price: 999
  }
test>
```

Step 3: Insert Multiple Documents

Query:

```
db.products.insertMany([
  { name: "phone", price: 500, stock: 10 },
  { name: "tablet", price: 300, stock: 5 },
  { name: "watch", price: 150, stock: 0 }
]);
```

Output:

```
> db.products.insertMany([
    { name: "phone", price: 500, stock: 10 },
    { name: "tablet", price: 300, stock: 5 },
    { name: "watch", price: 150, stock: 0 }
  ]);
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6ac6498b8fa08d8f4561c385'),
      '1': ObjectId('6ac6498b8fa08d8f4561c386'),
      '2': ObjectId('6ac6498b8fa08d8f4561c387')
    }
  }
test>
```

Step 4: View Products in Formatted JSON

Query:

```
db.products.find().pretty();
```

Output:

```
> db.products.find().pretty();
< {
    _id: ObjectId('6ac51e43d4bbf249c034346b'),
    name: 'phone',
    price: 500,
    stock: 10
  }
  {
    _id: ObjectId('6ac51e43d4bbf249c034346c'),
    name: 'tablet',
    price: 300,
    stock: 5
  }
  {
    _id: ObjectId('6ac51e43d4bbf249c034346d'),
    name: 'watch',
    price: 150,
    stock: 0
  }
  {
    _id: ObjectId('6ac6498b8fa08d8f4561c385'),
    name: 'phone',
    price: 500,
    stock: 10
  }
```

```
  {
    _id: ObjectId('6ac6498b8fa08d8f4561c386'),
    name: 'tablet',
    price: 300,
    stock: 5
  }
  {
    _id: ObjectId('6ac6498b8fa08d8f4561c387'),
    name: 'watch',
    price: 150,
    stock: 0
  }
test>
```

Name: Aryan Sunil Bendre
Student: 5144984
Roll NO: 03
College: B. K. Birla College

Practical 4

Python program demonstrating Noise Elimination, Feature Selection, and Exploratory Data Analysis (EDA) using Pandas, NumPy, Matplotlib, and Scikit-learn.

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_selection import VarianceThreshold

# ----------------------------
# Sample Dataset
# ----------------------------
data = {
    'Age': [20, 21, 22, 23, 24, 100],   # 100 is an outlier (noise)
    'Marks': [75, 80, 85, 90, 95, 20],
    'Attendance': [85, 88, 90, 92, 94, 50],
    'Constant': [1, 1, 1, 1, 1, 1]      # Constant feature
}

df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

# ====================================
# 1. Noise Elimination (Outlier Removal)
# ====================================
Q1 = df['Age'].quantile(0.25)
Q3 = df['Age'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df_clean = df[(df['Age'] >= lower) & (df['Age'] <= upper)]

print("\nDataset after Noise Elimination:")
print(df_clean)

# ====================================
# 2. Feature Selection
```

```python
# Remove constant or low-variance features
# ======================================
selector = VarianceThreshold(threshold=0.0)
selected = selector.fit_transform(df_clean)

selected_columns = df_clean.columns[selector.get_support()]

print("\nSelected Features:")
print(selected_columns)

# ======================================
# 3. Exploratory Data Analysis (EDA)
# ======================================

print("\nDataset Information:")
print(df_clean.info())

print("\nStatistical Summary:")
print(df_clean.describe())

print("\nCorrelation Matrix:")
print(df_clean.corr())

# Histogram
df_clean.hist(figsize=(8,6))
plt.suptitle("Histogram of Features")
plt.show()

# Boxplot
df_clean.boxplot(figsize=(8,5))
plt.title("Boxplot of Dataset")
plt.show()
```

```
Original Dataset:
   Age  Marks  Attendance  Constant
0   20     75          85         1
1   21     80          88         1
2   22     85          90         1
3   23     90          92         1
4   24     95          94         1
5  100     20          50         1

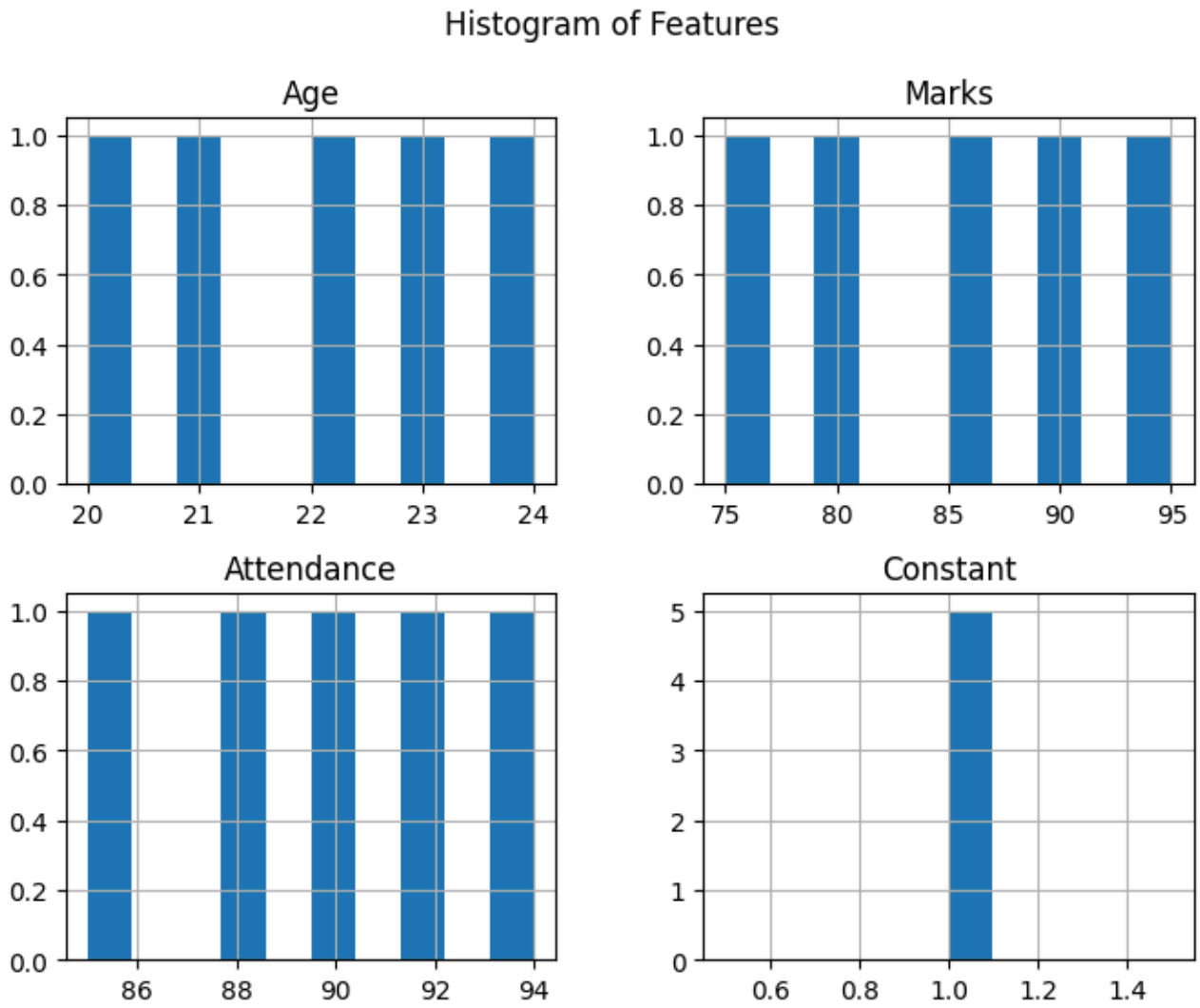
Dataset after Noise Elimination:
   Age  Marks  Attendance  Constant
0   20     75          85         1
1   21     80          88         1
2   22     85          90         1
3   23     90          92         1
4   24     95          94         1

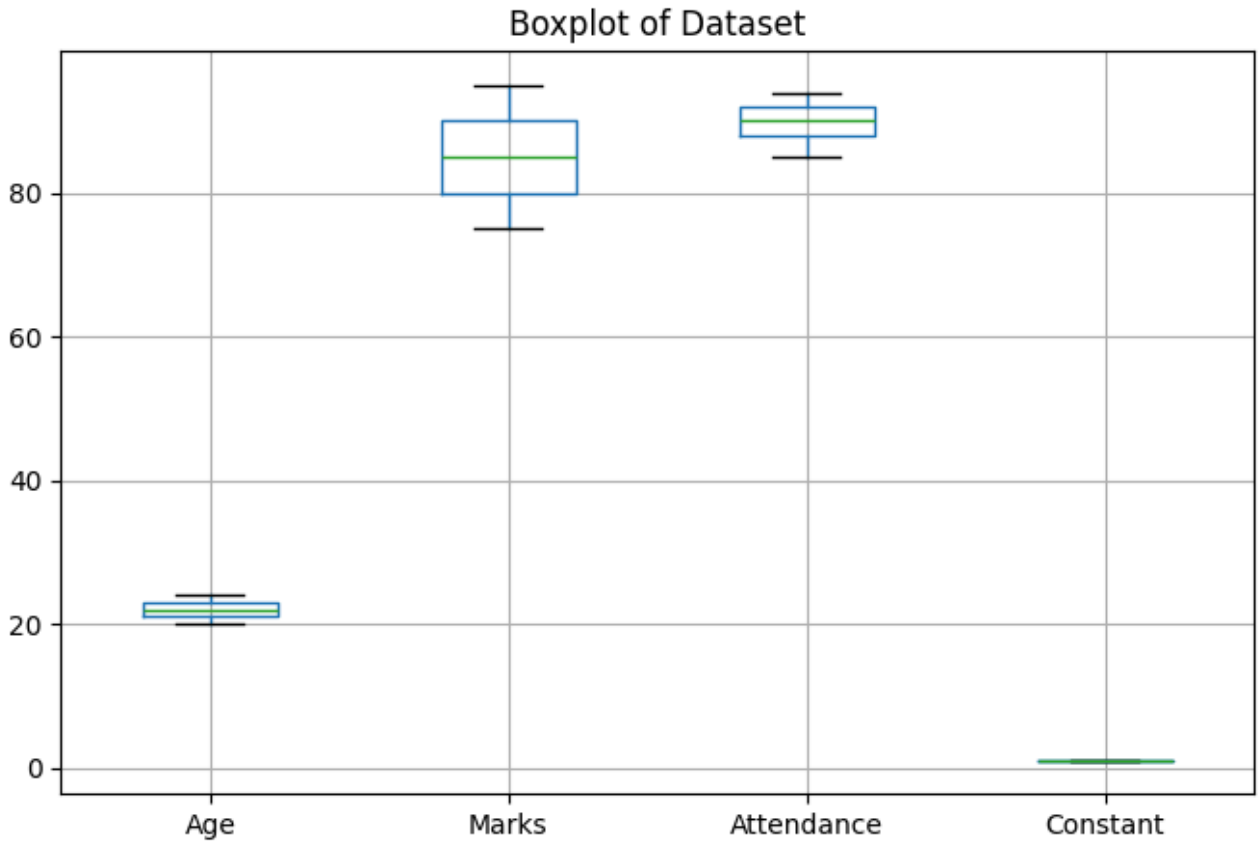
Selected Features:
Index(['Age', 'Marks', 'Attendance'], dtype='object')

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
Index: 5 entries, 0 to 4
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Age         5 non-null      int64
 1   Marks       5 non-null      int64
 2   Attendance  5 non-null      int64
 3   Constant    5 non-null      int64
dtypes: int64(4)
memory usage: 200.0 bytes
None

Statistical Summary:
             Age      Marks  Attendance  Constant
count   5.000000   5.000000     5.00000       5.0
mean   22.000000  85.000000    89.80000       1.0
std     1.581139   7.905694     3.49285       0.0
min    20.000000  75.000000    85.00000       1.0
25%    21.000000  80.000000    88.00000       1.0
50%    22.000000  85.000000    90.00000       1.0
75%    23.000000  90.000000    92.00000       1.0
max    24.000000  95.000000    94.00000       1.0

Correlation Matrix:
                 Age     Marks  Attendance  Constant
Age         1.000000  1.000000    0.995893       NaN
Marks       1.000000  1.000000    0.995893       NaN
Attendance  0.995893  0.995893    1.000000       NaN
Constant         NaN       NaN         NaN       NaN
```


Histogram of Features
Age
Marks
Attendance
Constant


Boxplot of Dataset
80
60
40
20
0
Age
Marks
Attendance
Constant

Name: Aryan Sunil Bendre
Student: 5144984
Roll NO: 03
College: B. K. Birla College

Practical 5

Extracting Data from APIs and Flat Files Using Python1. ObjectiveWrite a Python script to fetch user data from a public REST API, clean it, and merge it with a flat CSV file containing location details.2. Code ImplementationCreate a script named data_extraction.

In [ ]:
```python
import pandas as pd
import requests


def extract_api_data(url):
    """Fetches data from a REST API and returns a DataFrame."""
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        data = response.json()
        # Normalizing JSON data into a flat table
        return pd.json_normalize(data)
    except requests.exceptions.RequestException as e:
        print(f"API Error: {e}")
        return pd.DataFrame()


def extract_csv_data(file_path):
    """Reads a flat CSV file."""
    try:
        return pd.read_csv(file_path)
    except FileNotFoundError as e:
        print(f"File Error: {e}")
        return pd.DataFrame()


if __name__ == "__main__":
    # 1. API Extraction
    # Using jsonplaceholder to get mock users API data
    api_url = "https://jsonplaceholder.typicode.com/users"
    api_df = extract_api_data(api_url)

    if not api_df.empty:
        api_df = api_df[["id", "name", "email", "company.name"]]
```

```
        api_df.rename(columns={"company.name": "company"}, inplace=True)

    # 2. Flat File Extraction (Mock CSV Data)
    csv_file = "locations.csv"

    # Fixed the missing list of IDs to resolve the SyntaxError
    pd.DataFrame(
        {
            "id": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
            "city": [
                "New York",
                "London",
                "Paris",
                "Tokyo",
                "Berlin",
                "Delhi",
                "Sydney",
                "Moscow",
                "Cairo",
                "Beijing",
            ],
        }
    ).to_csv(csv_file, index=False)

    csv_df = extract_csv_data(csv_file)

    # 3. Data Transformation & Merging
    if not api_df.empty and not csv_df.empty:
        merged_df = pd.merge(api_df, csv_df, on="id", how="inner")
        print("\n--- Merged ETL Pipeline Data View ---")
        print(merged_df.head())

        # Save to target Data Lake / Warehouse stage
        merged_df.to_csv("cleaned_warehouse_profiles.csv", index=False)
        print("\nData successfully saved to 'cleaned_warehouse_profiles.c
```

```
--- Merged ETL Pipeline Data View ---
   id              name                      email             company  \
0   1     Leanne Graham          Sincere@april.biz     Romaguera-Crona
1   2      Ervin Howell          Shanna@melissa.tv        Deckow-Crist
2   3  Clementine Bauch         Nathan@yesenia.net  Romaguera-Jacobson
3   4  Patricia Lebsack  Julianne.OConner@kory.org       Robel-Corkery
4   5  Chelsey Dietrich   Lucio_Hettinger@annie.ca         Keebler LLC

       city
0  New York
1    London
2     Paris
3     Tokyo
4    Berlin

Data successfully saved to 'cleaned_warehouse_profiles.csv'
```

Name: Aryan Sunil Bendre
Student: 5144984
Roll NO: 03
College: B. K. Birla College

Practical 6

Setting up Apache Airflow and Creating a DAG1. ObjectiveConfigure a Directed Acyclic Graph (DAG) in Apache Airflow to run daily, executing sequential dependency tasks: a start notifier, an extraction script, and a pipeline success logger.2. DAG ImplementationPlace this code in your Airflow DAGs directory (usually ~/airflow/dags/data_pipeline_dag.py):pythonfrom datetime import datetime, timedelta

In [ ]:
```
!pip install apache-airflow
```

```
Requirement already satisfied: apache-airflow in /usr/local/lib/python3.1
3/dist-packages (3.3.2)
Requirement already satisfied: apache-airflow-core==3.3.2 in /usr/local/l
ib/python3.13/dist-packages (from apache-airflow) (3.3.2)
Requirement already satisfied: apache-airflow-task-sdk==1.3.2 in /usr/loc
al/lib/python3.13/dist-packages (from apache-airflow) (1.3.2)
Requirement already satisfied: a2wsgi>=1.10.8 in /usr/local/lib/python3.1
3/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (1.10.1
0)
Requirement already satisfied: aiosqlite!=0.22.0,>=0.20.0 in /usr/local/l
ib/python3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airf
low) (0.22.1)
Requirement already satisfied: alembic<2.0,>=1.13.1 in /usr/local/lib/pyt
hon3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(1.20.0)
Requirement already satisfied: apache-airflow-providers-common-compa
t>=1.7.4 in /usr/local/lib/python3.13/dist-packages (from apache-airflow-
core==3.3.2->apache-airflow) (1.20.0)
Requirement already satisfied: apache-airflow-providers-common-io>=1.6.3
in /usr/local/lib/python3.13/dist-packages (from apache-airflow-cor
e==3.3.2->apache-airflow) (1.10.0)
Requirement already satisfied: apache-airflow-providers-common-sql>=1.2
8.1 in /usr/local/lib/python3.13/dist-packages (from apache-airflow-cor
e==3.3.2->apache-airflow) (2.2.0)
Requirement already satisfied: apache-airflow-providers-smtp>=2.3.1 in /u
sr/local/lib/python3.13/dist-packages (from apache-airflow-core==3.3.2->a
pache-airflow) (3.1.0)
Requirement already satisfied: apache-airflow-providers-standard>=1.9.0 i
n /usr/local/lib/python3.13/dist-packages (from apache-airflow-cor
e==3.3.2->apache-airflow) (1.20.0)
Requirement already satisfied: argcomplete>=1.10 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(3.7.2)
Requirement already satisfied: asgiref>=2.3.0 in /usr/local/lib/python3.1
3/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (3.1
2.1)
Requirement already satisfied: attrs!=25.2.0,>=22.1.0 in /usr/local/lib/p
ython3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(26.1.0)
Requirement already satisfied: cachetools>=6.0.0 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(6.2.6)
Requirement already satisfied: cadwyn>=6.1.1 in /usr/local/lib/python3.1
3/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (7.0.0)
Requirement already satisfied: colorlog>=6.8.2 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (6.1
2.0)
Requirement already satisfied: cron-descriptor>=1.2.24 in /usr/local/lib/
python3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflo
w) (2.1.1)
Requirement already satisfied: croniter>=6.2.2 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(6.2.4)
Requirement already satisfied: cryptography>=44.0.3 in /usr/local/lib/pyt
hon3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
```

```
(50.0.2)
Requirement already satisfied: deprecated>=1.2.13 in /usr/local/lib/pytho
n3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(3.0.0)
Requirement already satisfied: dill>=0.2.2 in /usr/local/lib/python3.13/d
ist-packages (from apache-airflow-core==3.3.2->apache-airflow) (0.4.1)
Requirement already satisfied: fastapi<0.137.0,>=0.129.0 in /usr/local/li
b/python3.13/dist-packages (from fastapi[standard-no-fastapi-cloud-cl
i]<0.137.0,>=0.129.0->apache-airflow-core==3.3.2->apache-airflow) (0.13
6.3)
Requirement already satisfied: httpx>=0.25.0 in /usr/local/lib/python3.1
3/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (0.2
8.1)
Requirement already satisfied: importlib-metadata>=7.0 in /usr/local/lib/
python3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflo
w) (9.0.1)
Requirement already satisfied: isoduration>=20.11.0 in /usr/local/lib/pyt
hon3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(20.11.0)
Requirement already satisfied: itsdangerous>=2.0 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(2.2.0)
Requirement already satisfied: jinja2>=3.1.5 in /usr/local/lib/python3.1
3/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (3.1.6)
Requirement already satisfied: jsonschema>=4.19.1 in /usr/local/lib/pytho
n3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(4.26.0)
Requirement already satisfied: lazy-object-proxy>=1.2.0 in /usr/local/li
b/python3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airfl
ow) (1.12.0)
Requirement already satisfied: libcst>=1.8.2 in /usr/local/lib/python3.1
3/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (1.9.0)
Requirement already satisfied: linkify-it-py>=2.0.0 in /usr/local/lib/pyt
hon3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(2.2.0)
Requirement already satisfied: lockfile>=0.12.2 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (0.1
2.2)
Requirement already satisfied: methodtools>=0.4.7 in /usr/local/lib/pytho
n3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(0.4.7)
Requirement already satisfied: msgspec>=0.19.0 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (0.2
2.0)
Requirement already satisfied: natsort>=8.4.0 in /usr/local/lib/python3.1
3/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (8.4.0)
Requirement already satisfied: opentelemetry-api>=1.27.0 in /usr/local/li
b/python3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airfl
ow) (1.45.1)
Requirement already satisfied: opentelemetry-exporter-otlp>=1.27.0 in /us
r/local/lib/python3.13/dist-packages (from apache-airflow-core==3.3.2->ap
ache-airflow) (1.45.1)
Requirement already satisfied: opentelemetry-proto<9999,>=1.27.0 in /usr/
local/lib/python3.13/dist-packages (from apache-airflow-core==3.3.2->apac
he-airflow) (1.45.1)
```

```
Requirement already satisfied: packaging>=25.0 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (2
6.3)
Requirement already satisfied: pathspec>=0.9.0 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(1.1.1)
Requirement already satisfied: pendulum>=3.1.0 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(3.2.0)
Requirement already satisfied: pluggy>=1.5.0 in /usr/local/lib/python3.1
3/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (1.6.0)
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.1
3/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (7.2.2)
Requirement already satisfied: pydantic>=2.11.0 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (2.1
3.5)
Requirement already satisfied: pygments!=2.19.0,>=2.0.1 in /usr/local/li
b/python3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airfl
ow) (2.21.0)
Requirement already satisfied: pygtrie>=2.5.0 in /usr/local/lib/python3.1
3/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (2.6.2)
Requirement already satisfied: pyjwt>=2.11.0 in /usr/local/lib/python3.1
3/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (2.1
5.1)
Requirement already satisfied: python-daemon>=3.0.0 in /usr/local/lib/pyt
hon3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(3.1.2)
Requirement already satisfied: python-dateutil>=2.7.0 in /usr/local/lib/p
ython3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(2.9.0.post0)
Requirement already satisfied: python-slugify>=5.0 in /usr/local/lib/pyth
on3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(9.1.2)
Requirement already satisfied: pyyaml>=6.0.3 in /usr/local/lib/python3.1
3/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (6.0.3)
Requirement already satisfied: requests<3,>=2.32.0 in /usr/local/lib/pyth
on3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(2.32.4)
Requirement already satisfied: rich-argparse>=1.0.0 in /usr/local/lib/pyt
hon3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(1.8.0)
Requirement already satisfied: rich>=13.6.0 in /usr/local/lib/python3.13/
dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (13.9.4)
Requirement already satisfied: setproctitle>=1.3.3 in /usr/local/lib/pyth
on3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(1.3.8)
Requirement already satisfied: sqlalchemy>=2.0.50 in /usr/local/lib/pytho
n3.13/dist-packages (from sqlalchemy[asyncio]>=2.0.50->apache-airflow-cor
e==3.3.2->apache-airflow) (2.1.1)
Requirement already satisfied: starlette>=1.0.1 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(1.7.0)
Requirement already satisfied: structlog>=26.1.0 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (2
6.1.0)
```

```
Requirement already satisfied: svcs>=25.1.0 in /usr/local/lib/python3.13/
dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (26.2.0)
Requirement already satisfied: tabulate>=0.9.0 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(0.9.0)
Requirement already satisfied: tenacity>=8.3.0 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(9.1.4)
Requirement already satisfied: termcolor>=3.0.0 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow)
(3.3.0)
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/li
b/python3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airfl
ow) (4.16.0)
Requirement already satisfied: universal-pathlib>=0.3.8 in /usr/local/li
b/python3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airfl
ow) (0.3.10)
Requirement already satisfied: uuid6>=2024.7.10 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (202
5.0.1)
Requirement already satisfied: uvicorn>=0.37.0 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-core==3.3.2->apache-airflow) (0.5
4.0)
Requirement already satisfied: babel>=2.17.0 in /usr/local/lib/python3.1
3/dist-packages (from apache-airflow-task-sdk==1.3.2->apache-airflow)
(2.18.0)
Requirement already satisfied: fsspec>=2023.10.0 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-task-sdk==1.3.2->apache-airflow)
(2025.12.0)
Requirement already satisfied: greenback>=1.2.1 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-task-sdk==1.3.2->apache-airflow)
(1.3.0)
Requirement already satisfied: Mako in /usr/lib/python3/dist-packages (fr
om alembic<2.0,>=1.13.1->apache-airflow-core==3.3.2->apache-airflow)
(1.3.2.dev0)
Requirement already satisfied: sqlparse>=0.5.1 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-providers-common-sql>=1.28.1->apa
che-airflow-core==3.3.2->apache-airflow) (0.6.0)
Requirement already satisfied: more-itertools>=9.0.0 in /usr/local/lib/py
thon3.13/dist-packages (from apache-airflow-providers-common-sql>=1.2
8.1->apache-airflow-core==3.3.2->apache-airflow) (10.8.0)
Requirement already satisfied: aiosmtplib>=0.1.6 in /usr/local/lib/python
3.13/dist-packages (from apache-airflow-providers-smtp>=2.3.1->apache-air
flow-core==3.3.2->apache-airflow) (5.1.3)
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/li
b/python3.13/dist-packages (from cadwyn>=6.1.1->apache-airflow-cor
e==3.3.2->apache-airflow) (0.4.4)
Requirement already satisfied: cffi>=2.0.0 in /usr/local/lib/python3.13/d
ist-packages (from cryptography>=44.0.3->apache-airflow-core==3.3.2->apac
he-airflow) (2.1.1)
Requirement already satisfied: wrapt<3,>=1.16 in /usr/local/lib/python3.1
3/dist-packages (from deprecated>=1.2.13->apache-airflow-core==3.3.2->apa
che-airflow) (2.5.0)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/pyt
hon3.13/dist-packages (from fastapi<0.137.0,>=0.129.0->fastapi[standard-n
```

```
o-fastapi-cloud-cli]<0.137.0,>=0.129.0->apache-airflow-core==3.3.2->apach
e-airflow) (0.0.5)
Requirement already satisfied: fastapi-cli>=0.0.8 in /usr/local/lib/pytho
n3.13/dist-packages (from fastapi-cli[standard-no-fastapi-cloud-cl
i]>=0.0.8; extra == "standard-no-fastapi-cloud-cli"->fastapi[standard-no-
fastapi-cloud-cli]<0.137.0,>=0.129.0->apache-airflow-core==3.3.2->apache-
airflow) (0.0.32)
Requirement already satisfied: python-multipart>=0.0.18 in /usr/local/li
b/python3.13/dist-packages (from fastapi[standard-no-fastapi-cloud-cl
i]<0.137.0,>=0.129.0->apache-airflow-core==3.3.2->apache-airflow) (0.0.3
2)
Requirement already satisfied: email-validator>=2.0.0 in /usr/local/lib/p
ython3.13/dist-packages (from fastapi[standard-no-fastapi-cloud-cli]<0.13
7.0,>=0.129.0->apache-airflow-core==3.3.2->apache-airflow) (2.3.0)
Requirement already satisfied: pydantic-settings>=2.0.0 in /usr/local/li
b/python3.13/dist-packages (from fastapi[standard-no-fastapi-cloud-cl
i]<0.137.0,>=0.129.0->apache-airflow-core==3.3.2->apache-airflow) (2.1
5.0)
Requirement already satisfied: pydantic-extra-types>=2.0.0 in /usr/local/
lib/python3.13/dist-packages (from fastapi[standard-no-fastapi-cloud-cl
i]<0.137.0,>=0.129.0->apache-airflow-core==3.3.2->apache-airflow) (2.1
1.1)
Requirement already satisfied: greenlet!=0.4.17 in /usr/local/lib/python
3.13/dist-packages (from greenback>=1.2.1->apache-airflow-task-sd
k==1.3.2->apache-airflow) (3.5.6)
Requirement already satisfied: sniffio in /usr/local/lib/python3.13/dist-
packages (from greenback>=1.2.1->apache-airflow-task-sdk==1.3.2->apache-a
irflow) (1.3.1)
Requirement already satisfied: outcome in /usr/local/lib/python3.13/dist-
packages (from greenback>=1.2.1->apache-airflow-task-sdk==1.3.2->apache-a
irflow) (1.3.0.post0)
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-pa
ckages (from httpx>=0.25.0->apache-airflow-core==3.3.2->apache-airflow)
(4.15.1)
Requirement already satisfied: certifi in /usr/local/lib/python3.13/dist-
packages (from httpx>=0.25.0->apache-airflow-core==3.3.2->apache-airflow)
(2026.7.22)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.1
3/dist-packages (from httpx>=0.25.0->apache-airflow-core==3.3.2->apache-a
irflow) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.13/dist-pac
kages (from httpx>=0.25.0->apache-airflow-core==3.3.2->apache-airflow)
(3.20)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dis
t-packages (from httpcore==1.*->httpx>=0.25.0->apache-airflow-cor
e==3.3.2->apache-airflow) (0.16.0)
Requirement already satisfied: zipp>=3.20 in /usr/local/lib/python3.13/di
st-packages (from importlib-metadata>=7.0->apache-airflow-core==3.3.2->ap
ache-airflow) (4.1.0)
Requirement already satisfied: arrow>=0.15.0 in /usr/local/lib/python3.1
3/dist-packages (from isoduration>=20.11.0->apache-airflow-core==3.3.2->a
pache-airflow) (1.4.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python
3.13/dist-packages (from jinja2>=3.1.5->apache-airflow-core==3.3.2->apach
e-airflow) (3.0.3)
```

```
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /u
sr/local/lib/python3.13/dist-packages (from jsonschema>=4.19.1->apache-ai
rflow-core==3.3.2->apache-airflow) (2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/pyth
on3.13/dist-packages (from jsonschema>=4.19.1->apache-airflow-cor
e==3.3.2->apache-airflow) (0.37.0)
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python
3.13/dist-packages (from jsonschema>=4.19.1->apache-airflow-cor
e==3.3.2->apache-airflow) (2026.6.3)
Requirement already satisfied: pyyaml-ft>=8.0.0 in /usr/local/lib/python
3.13/dist-packages (from libcst>=1.8.2->apache-airflow-core==3.3.2->apach
e-airflow) (8.0.0)
Requirement already satisfied: wirerope>=0.4.7 in /usr/local/lib/python
3.13/dist-packages (from methodtools>=0.4.7->apache-airflow-cor
e==3.3.2->apache-airflow) (1.0.0)
Requirement already satisfied: opentelemetry-exporter-otlp-proto-grp
c==1.45.1 in /usr/local/lib/python3.13/dist-packages (from opentelemetry-
exporter-otlp>=1.27.0->apache-airflow-core==3.3.2->apache-airflow) (1.4
5.1)
Requirement already satisfied: opentelemetry-exporter-otlp-proto-htt
p==1.45.1 in /usr/local/lib/python3.13/dist-packages (from opentelemetry-
exporter-otlp>=1.27.0->apache-airflow-core==3.3.2->apache-airflow) (1.4
5.1)
Requirement already satisfied: googleapis-common-protos~=1.57 in /usr/loc
al/lib/python3.13/dist-packages (from opentelemetry-exporter-otlp-proto-g
rpc==1.45.1->opentelemetry-exporter-otlp>=1.27.0->apache-airflow-cor
e==3.3.2->apache-airflow) (1.75.5)
Requirement already satisfied: grpcio<2.0.0,>=1.66.2 in /usr/local/lib/py
thon3.13/dist-packages (from opentelemetry-exporter-otlp-proto-grpc==1.4
5.1->opentelemetry-exporter-otlp>=1.27.0->apache-airflow-core==3.3.2->apa
che-airflow) (1.84.0)
Requirement already satisfied: opentelemetry-exporter-otlp-common==0.66b1
in /usr/local/lib/python3.13/dist-packages (from opentelemetry-exporter-o
tlp-proto-grpc==1.45.1->opentelemetry-exporter-otlp>=1.27.0->apache-airfl
ow-core==3.3.2->apache-airflow) (0.66b1)
Requirement already satisfied: opentelemetry-exporter-otlp-proto-commo
n==1.45.1 in /usr/local/lib/python3.13/dist-packages (from opentelemetry-
exporter-otlp-proto-grpc==1.45.1->opentelemetry-exporter-otlp>=1.27.0->ap
ache-airflow-core==3.3.2->apache-airflow) (1.45.1)
Requirement already satisfied: opentelemetry-sdk~=1.45.1 in /usr/local/li
b/python3.13/dist-packages (from opentelemetry-exporter-otlp-proto-grp
c==1.45.1->opentelemetry-exporter-otlp>=1.27.0->apache-airflow-cor
e==3.3.2->apache-airflow) (1.45.1)
Requirement already satisfied: protobuf<8.0,>=5.0 in /usr/local/lib/pytho
n3.13/dist-packages (from opentelemetry-proto<9999,>=1.27.0->apache-airfl
ow-core==3.3.2->apache-airflow) (6.33.6)
Requirement already satisfied: opentelemetry-exporter-http-transport==0.6
6b1 in /usr/local/lib/python3.13/dist-packages (from opentelemetry-export
er-http-transport[requests]==0.66b1->opentelemetry-exporter-otlp-proto-ht
tp==1.45.1->opentelemetry-exporter-otlp>=1.27.0->apache-airflow-cor
e==3.3.2->apache-airflow) (0.66b1)
Requirement already satisfied: tzdata>=2020.1 in /usr/local/lib/python3.1
3/dist-packages (from pendulum>=3.1.0->apache-airflow-core==3.3.2->apach
e-airflow) (2026.4)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/p
```

```
ython3.13/dist-packages (from pydantic>=2.11.0->apache-airflow-cor
e==3.3.2->apache-airflow) (0.8.0)
Requirement already satisfied: pydantic-core==2.46.5 in /usr/local/lib/py
thon3.13/dist-packages (from pydantic>=2.11.0->apache-airflow-cor
e==3.3.2->apache-airflow) (2.46.5)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dis
t-packages (from python-dateutil>=2.7.0->apache-airflow-core==3.3.2->apac
he-airflow) (1.17.0)
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/pyth
on3.13/dist-packages (from python-slugify>=5.0->apache-airflow-cor
e==3.3.2->apache-airflow) (1.3)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/li
b/python3.13/dist-packages (from requests<3,>=2.32.0->apache-airflow-cor
e==3.3.2->apache-airflow) (3.4.9)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/pytho
n3.13/dist-packages (from requests<3,>=2.32.0->apache-airflow-cor
e==3.3.2->apache-airflow) (2.5.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/py
thon3.13/dist-packages (from rich>=13.6.0->apache-airflow-core==3.3.2->ap
ache-airflow) (4.2.0)
Requirement already satisfied: pathlib-abc<0.6.0,>=0.5.1 in /usr/local/li
b/python3.13/dist-packages (from universal-pathlib>=0.3.8->apache-airflo
w-core==3.3.2->apache-airflow) (0.5.2)
Requirement already satisfied: click>=7.0 in /usr/local/lib/python3.13/di
st-packages (from uvicorn>=0.37.0->apache-airflow-core==3.3.2->apache-air
flow) (8.5.0)
Requirement already satisfied: pycparser in /usr/local/lib/python3.13/dis
t-packages (from cffi>=2.0.0->cryptography>=44.0.3->apache-airflow-cor
e==3.3.2->apache-airflow) (3.0)
Requirement already satisfied: dnspython>=2.0.0 in /usr/local/lib/python
3.13/dist-packages (from email-validator>=2.0.0->fastapi[standard-no-fast
api-cloud-cli]<0.137.0,>=0.129.0->apache-airflow-core==3.3.2->apache-airf
low) (2.8.0)
Requirement already satisfied: typer>=0.16.0 in /usr/local/lib/python3.1
3/dist-packages (from fastapi-cli>=0.0.8->fastapi-cli[standard-no-fastap
i-cloud-cli]>=0.0.8; extra == "standard-no-fastapi-cloud-cli"->fastapi[st
andard-no-fastapi-cloud-cli]<0.137.0,>=0.129.0->apache-airflow-cor
e==3.3.2->apache-airflow) (0.27.2)
Requirement already satisfied: rich-toolkit>=0.14.8 in /usr/local/lib/pyt
hon3.13/dist-packages (from fastapi-cli>=0.0.8->fastapi-cli[standard-no-f
astapi-cloud-cli]>=0.0.8; extra == "standard-no-fastapi-cloud-cli"->fasta
pi[standard-no-fastapi-cloud-cli]<0.137.0,>=0.129.0->apache-airflow-cor
e==3.3.2->apache-airflow) (0.20.6)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.13/di
st-packages (from markdown-it-py>=2.2.0->rich>=13.6.0->apache-airflow-cor
e==3.3.2->apache-airflow) (0.1.2)
Requirement already satisfied: python-dotenv>=0.21.0 in /usr/local/lib/py
thon3.13/dist-packages (from pydantic-settings>=2.0.0->fastapi[standard-n
o-fastapi-cloud-cli]<0.137.0,>=0.129.0->apache-airflow-core==3.3.2->apach
e-airflow) (1.2.3)
Requirement already satisfied: httptools>=0.8.0 in /usr/local/lib/python
3.13/dist-packages (from uvicorn[standard]>=0.12.0; extra == "standard-n
o-fastapi-cloud-cli"->fastapi[standard-no-fastapi-cloud-cli]<0.13
7.0,>=0.129.0->apache-airflow-core==3.3.2->apache-airflow) (0.8.0)
Requirement already satisfied: uvloop>=0.15.1 in /usr/local/lib/python3.1
```

```
3/dist-packages (from uvicorn[standard]>=0.12.0; extra == "standard-no-fa
stapi-cloud-cli"->fastapi[standard-no-fastapi-cloud-cli]<0.137.0,>=0.12
9.0->apache-airflow-core==3.3.2->apache-airflow) (0.22.1)
Requirement already satisfied: watchfiles>=0.20 in /usr/local/lib/python
3.13/dist-packages (from uvicorn[standard]>=0.12.0; extra == "standard-n
o-fastapi-cloud-cli"->fastapi[standard-no-fastapi-cloud-cli]<0.13
7.0,>=0.129.0->apache-airflow-core==3.3.2->apache-airflow) (1.3.0)
Requirement already satisfied: websockets>=13.0 in /usr/local/lib/python
3.13/dist-packages (from uvicorn[standard]>=0.12.0; extra == "standard-n
o-fastapi-cloud-cli"->fastapi[standard-no-fastapi-cloud-cli]<0.13
7.0,>=0.129.0->apache-airflow-core==3.3.2->apache-airflow) (15.0.1)
Requirement already satisfied: opentelemetry-semantic-conventions==0.66b1
in /usr/local/lib/python3.13/dist-packages (from opentelemetry-sdk~=1.4
5.1->opentelemetry-exporter-otlp-proto-grpc==1.45.1->opentelemetry-export
er-otlp>=1.27.0->apache-airflow-core==3.3.2->apache-airflow) (0.66b1)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/pytho
n3.13/dist-packages (from typer>=0.16.0->fastapi-cli>=0.0.8->fastapi-cl
i[standard-no-fastapi-cloud-cli]>=0.0.8; extra == "standard-no-fastapi-cl
oud-cli"->fastapi[standard-no-fastapi-cloud-cli]<0.137.0,>=0.129.0->apach
e-airflow-core==3.3.2->apache-airflow) (1.5.4)
```

In [3]:

```python
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.empty import EmptyOperator

# Default arguments for the workflow pipeline execution
default_args = {
    "owner": "data_engineering_lab",
    "depends_on_past": False,
    "start_date": datetime(2026, 1, 1),
    "email_on_failure": False,
    "email_on_retry": False,
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

# Define the DAG context
with DAG(
    "university_etl_orchestration",
    default_args=default_args,
    description="Lab assignment for API and Flat File ETL orchestration",
    schedule=timedelta(days=1),
    catchup=False,
) as dag:

    # Task 1: Pipeline Init Anchor
    start_pipeline = EmptyOperator(task_id="start_pipeline")

    # Task 2: Run Python Extraction Script
    execute_extraction = BashOperator(
        task_id="run_extraction_script",
        bash_command="python3 /absolute/path/to/data_extraction.py",
    )
```

```
    # Task 3: Finalize and Log Metrics
    pipeline_complete = BashOperator(
        task_id="log_pipeline_success",
        bash_command='echo "ETL Execution completed successfully at $(dat
    )
```

```
2026-10-08T19:08:31.326406Z [warning  ] The `airflow.operators.bash.BashO
perator` attribute is deprecated. Please use `'airflow.providers.standar
d.operators.bash.BashOperator'`. [py.warnings] category=DeprecatedImportW
arning filename=/tmp/ipykernel_2342/851226352.py lineno=3
2026-10-08T19:08:31.331238Z [warning  ] The `airflow.operators.empty.Empt
yOperator` attribute is deprecated. Please use `'airflow.providers.standa
rd.operators.empty.EmptyOperator'`. [py.warnings] category=DeprecatedImpo
rtWarning filename=/tmp/ipykernel_2342/851226352.py lineno=4
```

Name: Aryan Sunil Bendre
Student: 5144984
Roll NO: 03
College: B. K. Birla College

Practical 7

1.Design an ETL pipeline to extract data from a CSV file, clean it, transform it, and store it in a database.

In [ ]:
```
import csv
import sqlite3
import os

# 1. SETUP: Create sample raw CSV
csv_file = 'raw_data.csv'
db_file = 'etl_task1.db'
with open(csv_file, 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['id', 'name', 'salary'])
    writer.writerow(['1', 'alice smith', '50000'])
    writer.writerow(['2', '', '40000']) # Missing name (needs cleaning)
    writer.writerow(['3', 'bob jones', '60000'])

print("--- Task 1: CSV ETL Pipeline ---")

# 2. EXTRACT
raw_data = []
with open(csv_file, 'r') as f:
    reader = csv.DictReader(f)
    raw_data = list(reader)

# 3. CLEAN & TRANSFORM
clean_data = []
for row in raw_data:
    if not row['name'].strip():
        continue # Clean: Skip missing names

    # Transform: Capitalize name and calculate a 10% tax column
    transformed_row = (
        int(row['id']),
        row['name'].title(),
        float(row['salary']),
        float(row['salary']) * 0.10
    )
    clean_data.append(transformed_row)

# 4. LOAD
```

```
conn = sqlite3.connect(db_file)
cursor = conn.cursor()
cursor.execute('''CREATE TABLE IF NOT EXISTS employees
                  (id INTEGER, name TEXT, salary REAL, tax REAL)''')
cursor.executemany('INSERT INTO employees VALUES (?, ?, ?, ?)', clean_dat
conn.commit()

# 5. VISIBLE OUTPUT
cursor.execute('SELECT * FROM employees')
print("Database contents after ETL:")
for row in cursor.fetchall():
    print(row)

# Cleanup
conn.close()
os.remove(csv_file)
os.remove(db_file)
```

```
--- Task 1: CSV ETL Pipeline ---
Database contents after ETL:
(1, 'Alice Smith', 50000.0, 5000.0)
(3, 'Bob Jones', 60000.0, 6000.0)
```

2.Extract data from multiple CSV files and combine them into a single dataset.

In [ ]:
```
import csv
import os

# 1. SETUP: Create multiple CSVs
files = ['region_a.csv', 'region_b.csv']
with open(files[0], 'w', newline='') as f:
    csv.writer(f).writerows([['id', 'product'], ['101', 'Laptop'], ['102'
with open(files[1], 'w', newline='') as f:
    csv.writer(f).writerows([['id', 'product'], ['103', 'Keyboard'], ['10

print("\n--- Task 2: Combine Multiple CSVs ---")

# 2. EXTRACT AND COMBINE
combined_dataset = []
header_extracted = False

for file in files:
    with open(file, 'r') as f:
        reader = csv.reader(f)
        header = next(reader)
        if not header_extracted:
            combined_dataset.append(header)
            header_extracted = True

        # Append data rows
        for row in reader:
            combined_dataset.append(row)
```

```
# 3. VISIBLE OUTPUT
print("Combined Dataset:")
for row in combined_dataset:
    print(row)

# Cleanup
for file in files:
    os.remove(file)
```

```
--- Task 2: Combine Multiple CSVs ---
Combined Dataset:
['id', 'product']
['101', 'Laptop']
['102', 'Mouse']
['103', 'Keyboard']
['104', 'Monitor']
```

3.Build a pipeline to extract JSON data, transform selected fields, and load it into a relational database.

In [ ]:
```
import json
import sqlite3
import os

# 1. SETUP: Create JSON file
json_file = 'orders.json'
db_file = 'etl_task3.db'
data = [
    {"order_id": "A1", "details": {"customer": "John", "total_cents": 250
    {"order_id": "A2", "details": {"customer": "Jane", "total_cents": 500
]
with open(json_file, 'w') as f:
    json.dump(data, f)

print("\n--- Task 3: JSON ETL Pipeline ---")

# 2. EXTRACT
with open(json_file, 'r') as f:
    json_data = json.load(f)

# 3. TRANSFORM
transformed_data = []
for item in json_data:
    # Flatten nested JSON, convert cents to dollars, format date string
    transformed_data.append((
        item['order_id'],
        item['details']['customer'],
        item['details']['total_cents'] / 100.0, # Transform to dollars
        item['date'].replace("-", "/")        # Transform date format
    ))

# 4. LOAD
conn = sqlite3.connect(db_file)
cursor = conn.cursor()
```

```
cursor.execute('''CREATE TABLE IF NOT EXISTS orders
                  (order_id TEXT, customer TEXT, total_dollars REAL, orde
cursor.executemany('INSERT INTO orders VALUES (?, ?, ?, ?)', transformed_
conn.commit()

# 5. VISIBLE OUTPUT
cursor.execute('SELECT * FROM orders')
print("Database contents after JSON ETL:")
for row in cursor.fetchall():
    print(row)

# Cleanup
conn.close()
os.remove(json_file)
os.remove(db_file)
```

```
--- Task 3: JSON ETL Pipeline ---
Database contents after JSON ETL:
('A1', 'John', 25.0, '2026/10/06')
('A2', 'Jane', 50.0, '2026/10/07')
```

4.Design an ETL process to identify and remove invalid records.

In [ ]:
```
print("\n--- Task 4: Remove Invalid Records ---")

# 1. EXTRACT (Simulated raw data)
raw_users = [
    {"user_id": 1, "age": 25, "email": "alice@test.com"},
    {"user_id": 2, "age": -5, "email": "bob@test.com"}, # Invalid age
    {"user_id": 3, "age": 30, "email": "charlie_no_at_symbol.com"}, # Inv
    {"user_id": 4, "age": 40, "email": "david@test.com"}
]

valid_records = []
invalid_records = []

# 2. CLEAN (Identify and split)
for user in raw_users:
    is_valid = True
    reason = []

    if user['age'] < 0 or user['age'] > 120:
        is_valid = False
        reason.append("Age out of bounds")

    if "@" not in user['email']:
        is_valid = False
        reason.append("Invalid email format")

    if is_valid:
        valid_records.append(user)
    else:
        user['error_reason'] = " | ".join(reason)
        invalid_records.append(user)
```

```
# 3. VISIBLE OUTPUT
print("Valid Records Retained:")
for r in valid_records:
    print(f"  {r}")

print("\nInvalid Records Removed:")
for r in invalid_records:
    print(f"  {r}")
```

```
--- Task 4: Remove Invalid Records ---
Valid Records Retained:
  {'user_id': 1, 'age': 25, 'email': 'alice@test.com'}
  {'user_id': 4, 'age': 40, 'email': 'david@test.com'}

Invalid Records Removed:
  {'user_id': 2, 'age': -5, 'email': 'bob@test.com', 'error_reason': 'Age
out of bounds'}
  {'user_id': 3, 'age': 30, 'email': 'charlie_no_at_symbol.com', 'error_r
eason': 'Invalid email format'}
```

5.Create a pipeline that performs data validation before loading data into the target database.

In [ ]:
```
import sqlite3
import os

print("\n--- Task 5: Pre-Load Validation ---")

db_file = 'etl_task5.db'
conn = sqlite3.connect(db_file)
cursor = conn.cursor()
cursor.execute('CREATE TABLE products (id INTEGER, name TEXT, price REAL)

incoming_data = [
    (1, "Table", 150.0),
    (2, "Chair", -20.0), # Fails price validation
    (3, None, 50.0)      # Fails name validation
]

# Validation Function
def validate_product(record):
    product_id, name, price = record
    if not name:
        return False, "Missing name"
    if price < 0:
        return False, "Negative price"
    return True, "Valid"

# Process and Load
successful_inserts = 0
for record in incoming_data:
    is_valid, msg = validate_product(record)
    if is_valid:
```

```
        cursor.execute('INSERT INTO products VALUES (?, ?, ?)', record)
        successful_inserts += 1
    else:
        print(f"Validation Failed for record {record}: {msg}")

conn.commit()

# VISIBLE OUTPUT
print(f"\nSuccessfully loaded {successful_inserts} valid records.")
cursor.execute('SELECT * FROM products')
for row in cursor.fetchall():
    print(row)

# Cleanup
conn.close()
os.remove(db_file)
```

```
--- Task 5: Pre-Load Validation ---
Validation Failed for record (2, 'Chair', -20.0): Negative price
Validation Failed for record (3, None, 50.0): Missing name

Successfully loaded 1 valid records.
(1, 'Table', 150.0)
```

6.Implement incremental data loading instead of loading the entire dataset every time.

In [ ]:
```
import sqlite3
import os

print("\n--- Task 6: Incremental Loading ---")

db_file = 'etl_task6.db'
conn = sqlite3.connect(db_file)
cursor = conn.cursor()

# 1. SETUP: Create table and load initial historical data
cursor.execute('''CREATE TABLE inventory
                  (item_id INTEGER PRIMARY KEY, item_name TEXT, qty INTEG
cursor.execute("INSERT INTO inventory VALUES (1, 'Pen', 100)")
cursor.execute("INSERT INTO inventory VALUES (2, 'Pencil', 200)")
conn.commit()

print("Initial Database State:")
for row in cursor.execute('SELECT * FROM inventory').fetchall():
    print(f"  {row}")

# 2. EXTRACT: New batch of data (Contains 1 update, 1 new item, 1 unchang
new_batch = [
    (1, 'Pen', 100),    # Unchanged
    (2, 'Pencil', 250), # Incremental Update (Quantity changed)
    (3, 'Eraser', 50)   # Incremental Insert (New item)
]
```

```python
# 3. INCREMENTAL LOAD (UPSERT Strategy)
# INSERT OR REPLACE automatically updates the row if the PRIMARY KEY (ite
cursor.executemany('''
    INSERT OR REPLACE INTO inventory (item_id, item_name, qty)
    VALUES (?, ?, ?)
''', new_batch)
conn.commit()

# 4. VISIBLE OUTPUT
print("\nDatabase State After Incremental Load (Notice ID 2 updated, ID 3
for row in cursor.execute('SELECT * FROM inventory').fetchall():
    print(f"  {row}")

# Cleanup
conn.close()
os.remove(db_file)
```

```
--- Task 6: Incremental Loading ---
Initial Database State:
  (1, 'Pen', 100)
  (2, 'Pencil', 200)

Database State After Incremental Load (Notice ID 2 updated, ID 3 added, n
o duplicates):
  (1, 'Pen', 100)
  (2, 'Pencil', 250)
  (3, 'Eraser', 50)
```

Name: Aryan Sunil Bendre
Student: 5144984
Roll NO: 03
College: B. K. Birla College

Practical 8

1.Write a PySpark program to read a CSV dataset and display its schema.

In [ ]:
```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, round
import os

# ========================================
# 0. SETUP: Initialize Spark and Mock Data
# ========================================
# Initialize a SparkSession
spark = SparkSession.builder \
    .appName("PySpark_Data_Operations") \
    .master("local[*]") \
    .getOrCreate()

# Create mock sales data (contains a duplicate for transaction_id 3)
with open("sales.csv", "w") as f:
    f.write("transaction_id,product_id,quantity,sale_amount\n")
    f.write("1,101,2,50.0\n")
    f.write("2,102,1,120.0\n")
    f.write("3,101,4,100.0\n")
    f.write("3,101,4,100.0\n")  # Duplicate row
    f.write("4,103,1,30.0\n")
    f.write("5,101,0,0.0\n")    # Invalid quantity to filter out

# Create mock products data
with open("products.csv", "w") as f:
    f.write("product_id,product_name,category\n")
    f.write("101,Wireless Mouse,Electronics\n")
    f.write("102,Mechanical Keyboard,Electronics\n")
    f.write("103,Coffee Mug,Home & Kitchen\n")

# ========================================
# 1. READ CSV & DISPLAY SCHEMA
# ========================================
print("\n--- 1. Reading Data and Displaying Schema ---")
# Read CSV files into DataFrames with schema inference
sales_df = spark.read.csv("sales.csv", header=True, inferSchema=True)
products_df = spark.read.csv("products.csv", header=True, inferSchema=Tru
```

```
# Display the schema of the sales DataFrame
sales_df.printSchema()
```

```
--- 1. Reading Data and Displaying Schema ---
root
 |-- transaction_id: integer (nullable = true)
 |-- product_id: integer (nullable = true)
 |-- quantity: integer (nullable = true)
 |-- sale_amount: double (nullable = true)
```

2.Implement a PySpark program to remove duplicate records.

In [ ]:
```
# 3. REMOVE DUPLICATE RECORDS
# ========================================
print("\n--- 2. Removing Duplicates ---")
print(f"Original row count: {sales_df.count()}")

# Drop duplicate rows across all columns
clean_sales_df = sales_df.dropDuplicates()

print(f"Row count after dropping duplicates: {clean_sales_df.count()}")
clean_sales_df.show()
```

```
--- 2. Removing Duplicates ---
Original row count: 6
Row count after dropping duplicates: 5
+--------------+----------+--------+-----------+
|transaction_id|product_id|quantity|sale_amount|
+--------------+----------+--------+-----------+
|             1|       101|       2|       50.0|
|             4|       103|       1|       30.0|
|             2|       102|       1|      120.0|
|             3|       101|       4|      100.0|
|             5|       101|       0|        0.0|
+--------------+----------+--------+-----------+
```

3.Perform filtering, grouping, and aggregation operations using PySpark DataFrames.

In [ ]:
```
# ========================================
# 2. FILTERING
# ========================================
print("\n--- 3. Filtering Operations ---")
# Filter out records where quantity is 0 or less
filtered_sales_df = clean_sales_df.filter(col("quantity") > 0)
filtered_sales_df.show()
```

```
--- 3. Filtering Operations ---
+--------------+----------+--------+-----------+
|transaction_id|product_id|quantity|sale_amount|
+--------------+----------+--------+-----------+
|             1|       101|       2|       50.0|
|             4|       103|       1|       30.0|
|             2|       102|       1|      120.0|
|             3|       101|       4|      100.0|
+--------------+----------+--------+-----------+
```

4.Perform a join between two datasets using PySpark.

In [ ]:
```
# 4. PERFORM A JOIN BETWEEN TWO DATASETS
# ========================================
print("\n--- 4. Joining DataFrames ---")
# Perform an inner join on 'product_id'
joined_df = filtered_sales_df.join(products_df, on="product_id", how="inn
joined_df.show()
```

```
--- 4. Joining DataFrames ---
+----------+--------------+--------+-----------+------------------
+--------------+
|product_id|transaction_id|quantity|sale_amount|       product_name|
category|
+----------+--------------+--------+-----------+------------------
+--------------+
|       101|             1|       2|       50.0|     Wireless Mouse|   El
ectronics|
|       103|             4|       1|       30.0|         Coffee Mug|Home
& Kitchen|
|       102|             2|       1|      120.0|Mechanical Keyboard|   El
ectronics|
|       101|             3|       4|      100.0|     Wireless Mouse|   El
ectronics|
+----------+--------------+--------+-----------+------------------
+--------------+
```

5.Write a PySpark program to calculate average sales by product category.

In [ ]:
```
# 5. GROUPING & AGGREGATION (Avg Sales by Category)
# ========================================
print("\n--- 5. Average Sales by Product Category ---")
# Group by category, calculate the average sale amount, and round to 2 de
avg_sales_df = joined_df.groupBy("category") \
    .agg(
        round(avg("sale_amount"), 2).alias("average_sales")
    )

avg_sales_df.show()
```

```python
# ==========================================
# CLEANUP
# ==========================================
# Stop the Spark session and remove mock files
spark.stop()
os.remove("sales.csv")
os.remove("products.csv")
```

```
--- 5. Average Sales by Product Category ---
+--------------+-------------+
|      category|average_sales|
+--------------+-------------+
|Home & Kitchen|         30.0|
|   Electronics|         90.0|
+--------------+-------------+
```

Name: Aryan Sunil Bendre
Student: 5144984
Roll NO: 03
College: B. K. Birla College

Practical 9

1.Build an end-to-end pipeline: CSV → Python/Pandas → Data Cleaning → SQL Database → Report.

In [ ]:
```
import pandas as pd
import sqlite3
import os

print("--- Task 1: End-to-End CSV to SQL Pipeline ---")

csv_file = 'raw_sales.csv'
db_file = 'task1_pipeline.db'

# 1. SETUP: Create messy raw data
with open(csv_file, 'w') as f:
    f.write("transaction_id,date,amount,category\n")
    f.write("1,2026-10-01,150.50,Electronics\n")
    f.write("2,,200.00,Clothing\n")          # Missing date
    f.write("3,2026-10-03,ERROR,Electronics\n") # Invalid amount string
    f.write("4,2026-10-04,99.99,Clothing\n")

# 2. EXTRACT: Read with Pandas
df = pd.read_csv(csv_file)

# 3. TRANSFORM & CLEAN
# Drop rows where 'date' is missing
df = df.dropna(subset=['date'])
# Convert 'amount' to numeric, forcing errors to NaN, then drop those NaN
df['amount'] = pd.to_numeric(df['amount'], errors='coerce')
df = df.dropna(subset=['amount'])
# Ensure proper date formatting
df['date'] = pd.to_datetime(df['date']).dt.strftime('%Y-%m-%d')

# 4. LOAD: Write to SQL Database
conn = sqlite3.connect(db_file)
df.to_sql('cleaned_sales', conn, if_exists='replace', index=False)

# 5. REPORT: Generate summary report using SQL
report_query = """
    SELECT category, COUNT(transaction_id) as total_sales, SUM(amount) as
    FROM cleaned_sales
    GROUP BY category
```

```
"""
report_df = pd.read_sql(report_query, conn)

print("\nFinal Aggregated Report:")
print(report_df.to_string(index=False))

# Cleanup
conn.close()
os.remove(csv_file)
os.remove(db_file)
```

```
--- Task 1: End-to-End CSV to SQL Pipeline ---

Final Aggregated Report:
   category  total_sales  total_revenue
   Clothing            1          99.99
Electronics            1         150.50
```

2.Build an ETL pipeline for an e-commerce dataset containing customers, orders, products, and payments.

In [ ]:
```
import pandas as pd
import sqlite3
import os

print("\n--- Task 2: E-Commerce ETL Pipeline ---")

db_file = 'task2_ecommerce.db'

# 1. EXTRACT: Simulated extraction of 4 different entities
customers = pd.DataFrame({
    'cust_id': [1, 2],
    'name': ['Alice', 'Bob']
})
products = pd.DataFrame({
    'prod_id': [101, 102],
    'price': [1200.0, 50.0]  # 101=Laptop, 102=Mouse
})
orders = pd.DataFrame({
    'order_id': [1001, 1002, 1003],
    'cust_id': [1, 1, 2],
    'prod_id': [101, 102, 102],
    'qty': [1, 2, 1]
})
payments = pd.DataFrame({
    'order_id': [1001, 1002, 1003],
    'status': ['Completed', 'Failed', 'Completed']
})

# 2. TRANSFORM
# Join Orders with Payments to filter only successful transactions
merged = orders.merge(payments, on='order_id')
successful_orders = merged[merged['status'] == 'Completed']
```

```
# Join with Products to calculate total transaction value
successful_orders = successful_orders.merge(products, on='prod_id')
successful_orders['total_value'] = successful_orders['qty'] * successful_

# Join with Customers to get names and aggregate revenue per customer
final_merged = successful_orders.merge(customers, on='cust_id')
customer_revenue = final_merged.groupby('name')['total_value'].sum().rese
customer_revenue.rename(columns={'total_value': 'lifetime_revenue'}, inpl

# 3. LOAD
conn = sqlite3.connect(db_file)
customer_revenue.to_sql('customer_revenue_report', conn, if_exists='repla

# 4. VISIBLE OUTPUT
print("\nFinal Customer Revenue Table (Notice Bob's failed order is exclu
print(pd.read_sql("SELECT * FROM customer_revenue_report", conn).to_strin

# Cleanup
conn.close()
os.remove(db_file)
```

```
--- Task 2: E-Commerce ETL Pipeline ---

Final Customer Revenue Table (Notice Bob's failed order is excluded):
 name  lifetime_revenue
Alice            1200.0
  Bob              50.0
```

3.Design a data pipeline that handles both historical and newly arriving data.

In [ ]:
```
import pandas as pd
import sqlite3
import os

print("\n--- Task 3: Historical & Arriving Data Pipeline ---")

db_file = 'task3_incremental.db'
conn = sqlite3.connect(db_file)
cursor = conn.cursor()

# 1. SETUP: Create table and insert HISTORICAL data
cursor.execute('''CREATE TABLE user_profiles
                  (user_id INTEGER PRIMARY KEY, email TEXT, status TEXT)'
historical_data = [
    (1, 'alice@test.com', 'Active'),
    (2, 'bob@test.com', 'Active')
]
cursor.executemany('INSERT INTO user_profiles VALUES (?, ?, ?)', historic
conn.commit()

print("Initial Historical Data in DB:")
print(pd.read_sql("SELECT * FROM user_profiles", conn).to_string(index=Fa
```

```python
# 2. EXTRACT: Newly arriving data (Contains an update for ID 2, and a new
new_data_df = pd.DataFrame({
    'user_id': [2, 3],
    'email': ['bob_updated@test.com', 'charlie@test.com'],
    'status': ['Inactive', 'Active']
})

# 3. TRANSFORM & LOAD: Incremental Upsert Strategy
# Step A: Load new data into a temporary staging table
new_data_df.to_sql('staging_profiles', conn, if_exists='replace', index=F

# Step B: Perform UPSERT (Insert or Replace) from staging to main table b
cursor.execute('''
    INSERT OR REPLACE INTO user_profiles (user_id, email, status)
    SELECT user_id, email, status FROM staging_profiles
''')
conn.commit()

# Step C: Drop staging table
cursor.execute('DROP TABLE staging_profiles')
conn.commit()

# 4. VISIBLE OUTPUT
print("\nDatabase State After Incremental Load (ID 2 Updated, ID 3 Insert
print(pd.read_sql("SELECT * FROM user_profiles", conn).to_string(index=Fa

# Cleanup
conn.close()
os.remove(db_file)
```

```
--- Task 3: Historical & Arriving Data Pipeline ---
Initial Historical Data in DB:
 user_id          email status
       1 alice@test.com Active
       2   bob@test.com Active

Database State After Incremental Load (ID 2 Updated, ID 3 Inserted):
 user_id               email   status
       1       alice@test.com   Active
       2 bob_updated@test.com Inactive
       3     charlie@test.com   Active
```

Name: Aryan Sunil Bendre Student: 5144984
Roll NO: 03
College: B. K. Birla College

Practical 10: End-to-End Data Engineering Mini Project

This comprehensive, standalone mini-project fulfills the requirements of Practical 10. While originally designed for a group of two, this scope has been expanded and adapted for a single developer to showcase a complete Data Engineering lifecycle.

The project simulates a retail company's pipeline. It includes:
1.Data Ingestion: Reading from diverse sources (CSV and JSON).
2.Data Cleaning & Transformation: Handling missing values, standardizing dates, and calculating derived metrics.
3.Data Warehouse Design: Implementing a Star Schema (Fact and Dimension tables).
4.Storage: Loading the transformed data into a relational SQLite database.
5.Reporting: Querying the warehouse to generate a business report and a data visualization.

Complete Project Code

It is completely self-contained; it will generate the necessary dummy raw data, process it through the entire pipeline, build the database, and output the final reports.

In [ ]:
```
import pandas as pd
import sqlite3
import json
import os
import matplotlib.pyplot as plt

print("=== STARTING END-TO-END DATA ENGINEERING PIPELINE ===")

# ==========================================
# PHASE 0: SETUP & RAW DATA GENERATION
# ==========================================
print("\n[Phase 0] Generating simulated raw data (CSVs and JSON)...")
```

```python
# 1. Customers Data (CSV)
customers_data = """customer_id,first_name,last_name,region
1,Alice,Smith,North
2,Bob,Jones,South
3,Charlie,Brown,East
4,Diana,Prince,West
5,Eve,,North""" # Intentional missing last name
with open("raw_customers.csv", "w") as f:
    f.write(customers_data)

# 2. Products Data (JSON)
products_data = [
    {"product_id": 101, "product_name": "Laptop", "category": "Electronic
    {"product_id": 102, "product_name": "Desk Chair", "category": "Furnit
    {"product_id": 103, "product_name": "Mouse", "category": "Electronics
    {"product_id": 104, "product_name": "Notebook", "category": "Stationе
]
with open("raw_products.json", "w") as f:
    json.dump(products_data, f)

# 3. Transactions Data (CSV) - Contains anomalies (missing dates, negativ
transactions_data = """tx_id,date,customer_id,product_id,quantity
1001,2026-10-01,1,101,1
1002,2026-10-02,2,102,2
1003,,3,103,1
1004,2026-10-02,1,103,2
1005,2026-10-03,4,104,-5
1006,2026-10-04,5,101,1"""
with open("raw_transactions.csv", "w") as f:
    f.write(transactions_data)


# ========================================
# PHASE 1: DATA INGESTION (EXTRACTION)
# ========================================
print("\n[Phase 1] Extracting data from multiple sources...")
df_customers = pd.read_csv("raw_customers.csv")
df_products = pd.read_json("raw_products.json")
df_transactions = pd.read_csv("raw_transactions.csv")


# ========================================
# PHASE 2: DATA CLEANING & TRANSFORMATION
# ========================================
print("\n[Phase 2] Cleaning and Transforming data...")

# Clean Customers: Fill missing last names, create full name
df_customers['last_name'] = df_customers['last_name'].fillna("Unknown")
df_customers['full_name'] = df_customers['first_name'] + " " + df_custome
df_customers = df_customers[['customer_id', 'full_name', 'region']] # Sel

# Clean Transactions: Remove invalid rows
initial_tx_count = len(df_transactions)
```

```
df_transactions = df_transactions.dropna(subset=['date']) # Drop missing
df_transactions = df_transactions[df_transactions['quantity'] > 0] # Remo
print(f"  -> Dropped {initial_tx_count - len(df_transactions)} invalid tr

# Transformation: Derive Date Dimension elements
df_transactions['date'] = pd.to_datetime(df_transactions['date'])
df_date_dim = pd.DataFrame({
    'date_id': df_transactions['date'].dt.strftime('%Y%m%d').astype(int),
    'full_date': df_transactions['date'].dt.date,
    'year': df_transactions['date'].dt.year,
    'month': df_transactions['date'].dt.month,
    'day': df_transactions['date'].dt.day
}).drop_duplicates()

# Transformation: Map date_id back to transactions and calculate Total Sa
df_transactions['date_id'] = df_transactions['date'].dt.strftime('%Y%m%d'
df_transactions = df_transactions.merge(df_products[['product_id', 'price
df_transactions['total_amount'] = df_transactions['quantity'] * df_transa

# Prepare Final Fact Table
df_fact_sales = df_transactions[['tx_id', 'date_id', 'customer_id', 'prod


# ========================================
# PHASE 3: DATA WAREHOUSE DESIGN & STORAGE
# ========================================
print("\n[Phase 3] Loading Star Schema into SQLite Data Warehouse...")
db_name = "enterprise_data_warehouse.db"
conn = sqlite3.connect(db_name)

# Load Dimensions
df_customers.to_sql("dim_customer", conn, if_exists="replace", index=Fals
df_products.to_sql("dim_product", conn, if_exists="replace", index=False)
df_date_dim.to_sql("dim_date", conn, if_exists="replace", index=False)

# Load Fact
df_fact_sales.to_sql("fact_sales", conn, if_exists="replace", index=False
print(f"  -> Successfully loaded {len(df_fact_sales)} records into fact_s


# ========================================
# PHASE 4: REPORTING & VISUALIZATION
# ========================================
print("\n[Phase 4] Generating Business Reports from Data Warehouse...")

# Report 1: Sales by Category (SQL JOIN across Star Schema)
query_sales_by_category = """
    SELECT
        p.category,
        SUM(f.quantity) as items_sold,
        SUM(f.total_amount) as total_revenue
    FROM fact_sales f
    JOIN dim_product p ON f.product_id = p.product_id
    GROUP BY p.category
```

```
    ORDER BY total_revenue DESC
"""
df_report1 = pd.read_sql(query_sales_by_category, conn)
print("\n--- REPORT 1: Revenue by Category ---")
print(df_report1.to_string(index=False))

# Report 2: Top Customers by Region
query_top_customers = """
    SELECT
        c.full_name,
        c.region,
        SUM(f.total_amount) as lifetime_value
    FROM fact_sales f
    JOIN dim_customer c ON f.customer_id = c.customer_id
    GROUP BY c.full_name, c.region
    ORDER BY lifetime_value DESC
"""
df_report2 = pd.read_sql(query_top_customers, conn)
print("\n--- REPORT 2: Top Customers ---")
print(df_report2.to_string(index=False))

# Visualization: Bar Chart of Sales by Category
plt.figure(figsize=(8, 5))
plt.bar(df_report1['category'], df_report1['total_revenue'], color=['#4C7
plt.title('Total Revenue by Product Category')
plt.xlabel('Category')
plt.ylabel('Revenue ($)')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
chart_filename = "revenue_report_chart.png"
plt.savefig(chart_filename)
print(f"\n  -> Visualization saved locally as '{chart_filename}'")


# ========================================
# CLEANUP
# ========================================
conn.close()
os.remove("raw_customers.csv")
os.remove("raw_products.json")
os.remove("raw_transactions.csv")
print("\n=== PIPELINE EXECUTION COMPLETE ===")
```

```
=== STARTING END-TO-END DATA ENGINEERING PIPELINE ===

[Phase 0] Generating simulated raw data (CSVs and JSON)...

[Phase 1] Extracting data from multiple sources...

[Phase 2] Cleaning and Transforming data...
  -> Dropped 2 invalid transaction records.

[Phase 3] Loading Star Schema into SQLite Data Warehouse...
  -> Successfully loaded 4 records into fact_sales.

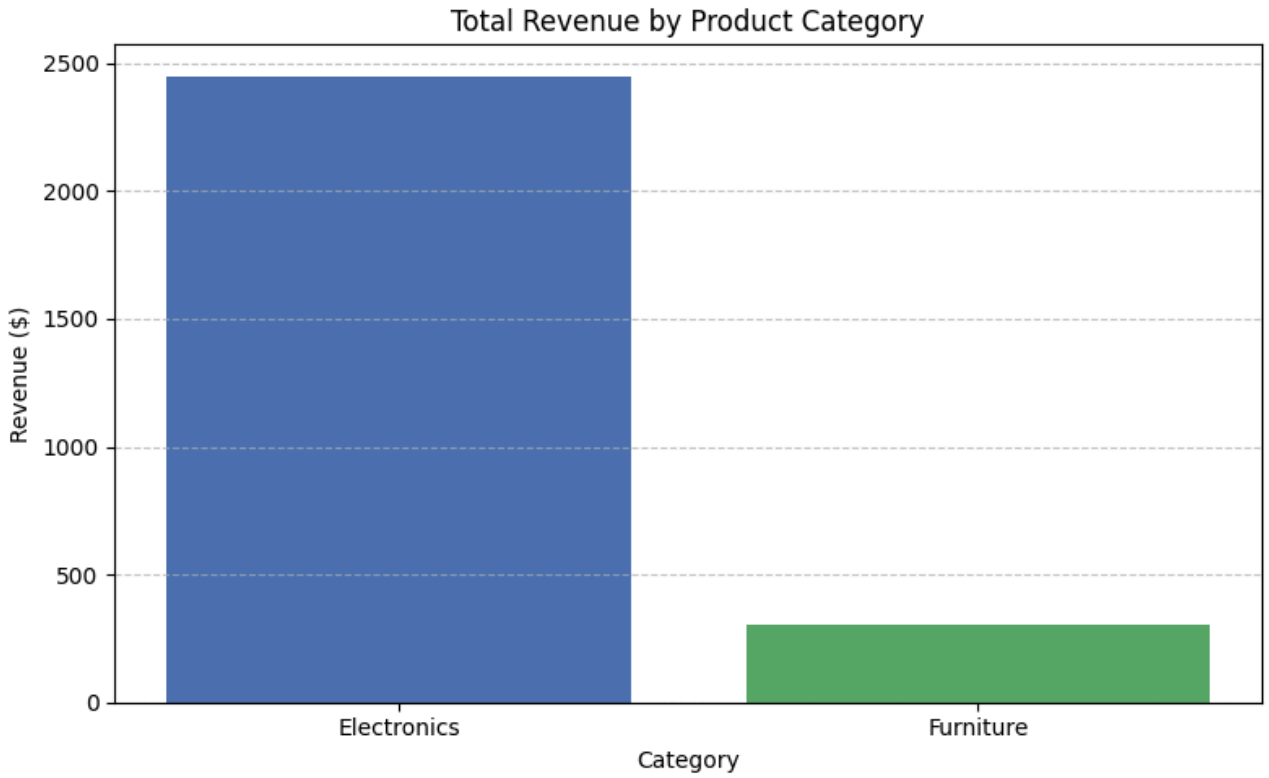
[Phase 4] Generating Business Reports from Data Warehouse...

--- REPORT 1: Revenue by Category ---
   category  items_sold  total_revenue
Electronics           4         2450.0
  Furniture           2          301.0

--- REPORT 2: Top Customers ---
  full_name region  lifetime_value
Alice Smith  North          1250.0
Eve Unknown  North          1200.0
  Bob Jones  South           301.0

  -> Visualization saved locally as 'revenue_report_chart.png'

=== PIPELINE EXECUTION COMPLETE ===
```



In [ ]:
```
# Optional: Export Star Schema to CSV for Power BI / Tableau
```

```
df_fact_sales.to_csv("fact_sales.csv", index=False)
df_customers.to_csv("dim_customer.csv", index=False)
df_products.to_csv("dim_product.csv", index=False)
df_date_dim.to_csv("dim_date.csv", index=False)
print("  -> Exported Star Schema to CSV for BI Tools.")
```

```
  -> Exported Star Schema to CSV for BI Tools.
```

Key Architectural Concepts Addressed

Data Ingestion: Handled mixed formats (CSV & JSON) and standardized them into Pandas DataFrames.

Data Cleaning: Removed corrupted transactions (null dates, negative quantities) and handled missing customer strings.

Data Warehouse Design (Star Schema): Modeled the data into a central fact_sales table surrounded by dim_customer, dim_product, and dim_date tables. This allows for highly optimized analytical querying.

Storage: Utilized an embedded SQLite database (enterprise_data_warehouse.db) to persist the architecture.

Power BI DashBoard

